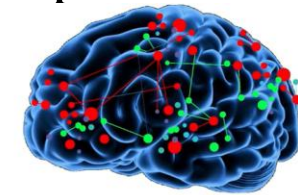


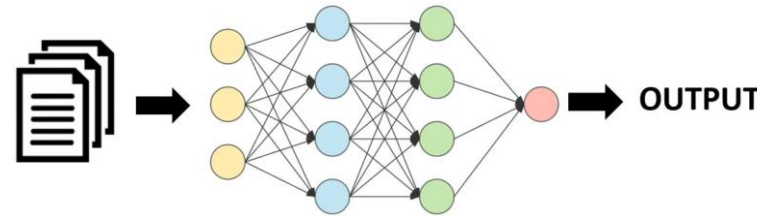
Artificial Neural Network (ANN)

ANN

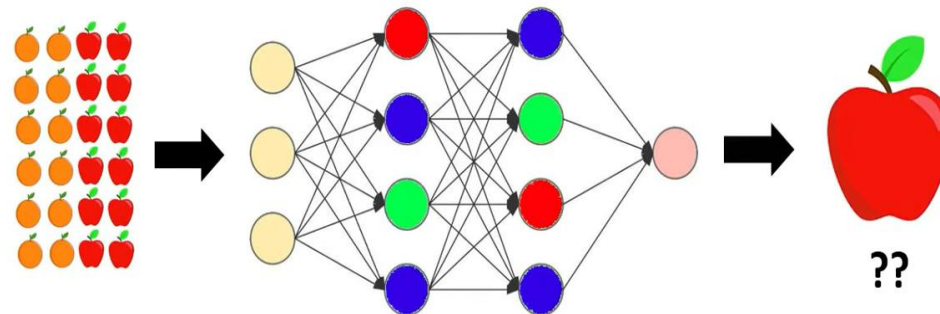
- The human brain has billions of neurons and trillions of connections between these neurons. With the help of this network of neurons, it always tries to recognize patterns in anything we see or experience.
- Let's say a baby is learning about the fruits. At first, it does not know about any kind of fruit but if he sees an image of a fruit, let's say apple for a certain number of times, its brain starts forming patterns inside which helps to recognize the apple next time he sees it.



- Similarly, the artificial neural network works. We feed a set of input data and based on this input data, the network tries to recognize patterns in it and makes output prediction for the new data.

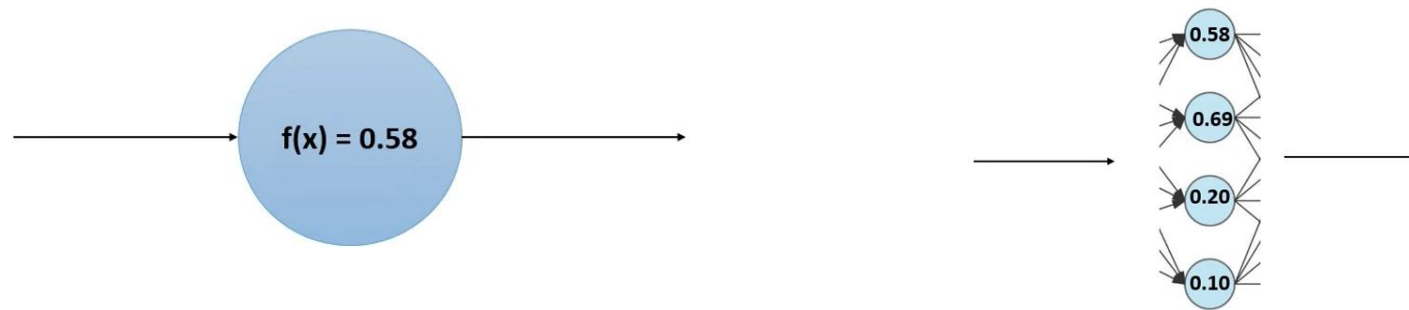


- For example, if we pass a bunch of images of apple and oranges, this network will try to recognize patterns in these images and based on this pattern stored, it will make the new predictions with the image it has never seen before.

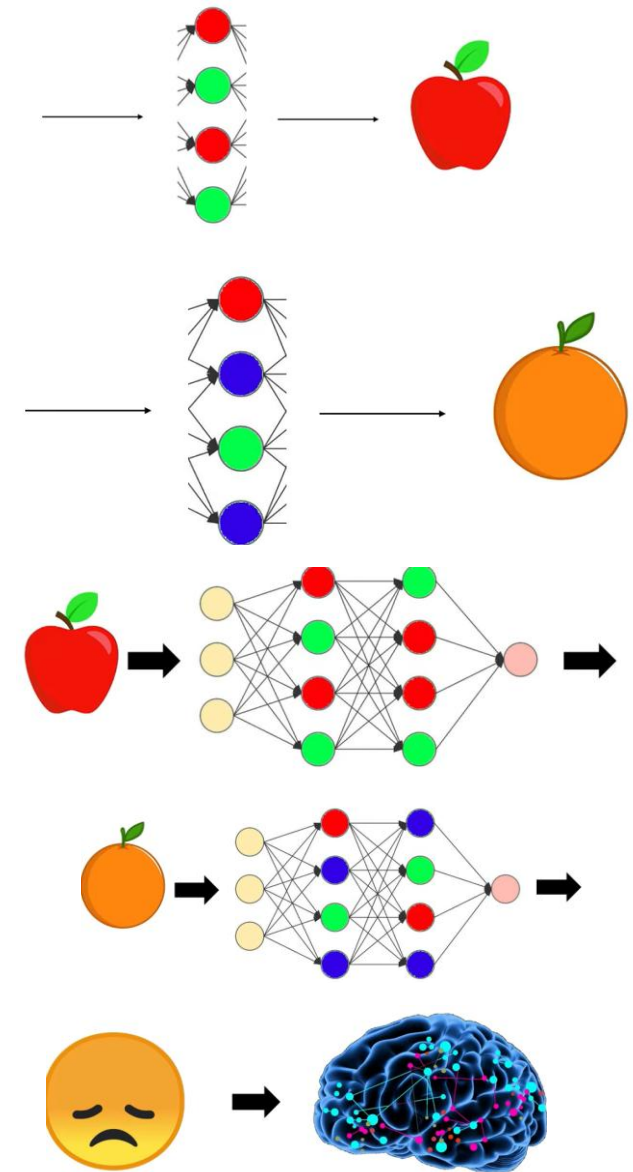


Neuron

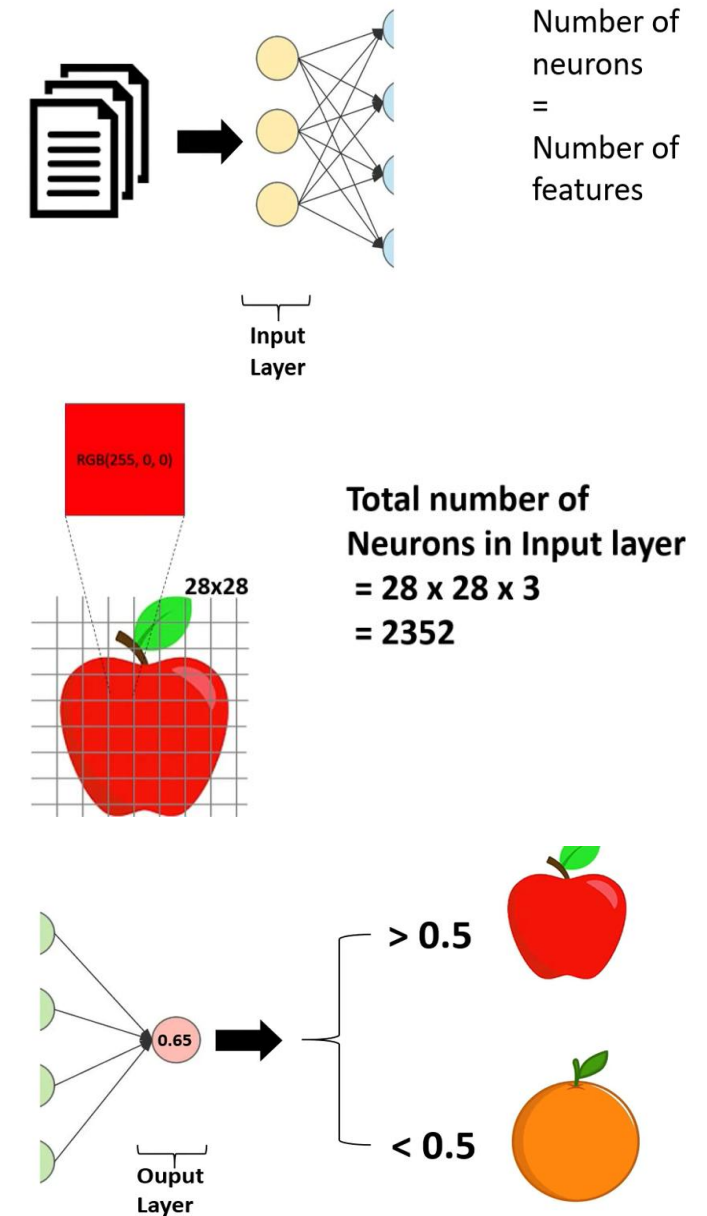
- Neurons are the function that gives some output value. This output value can be anything but it's usually small and between 0 to 1.
- The neuron as something that stores a value between 0 to 1.
- Now, different neurons stores different values in them and these different values are responsible for recognizing different patterns at the different region.



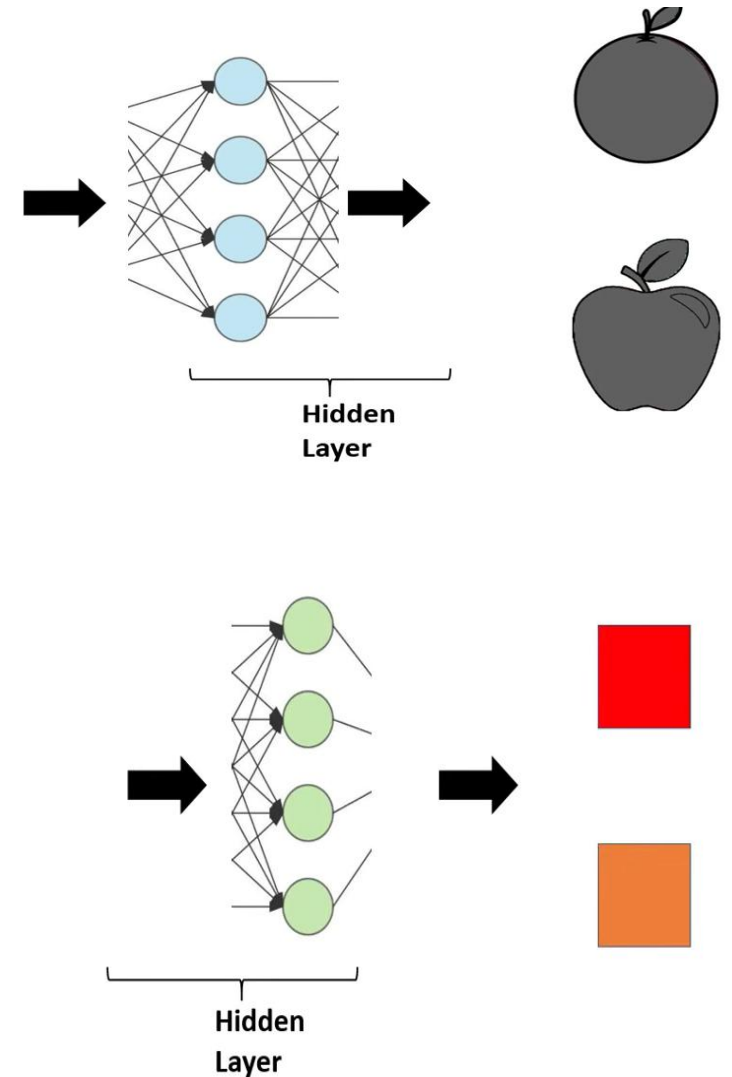
- For example, there may be some neurons which hold some numbers that are responsible for recognizing the red color in an image of an apple.
- And there may be some other neurons for recognizing the orange color in the image of an orange.
- So, you can say that when we feed an image of an apple, some neurons get activated and when we feed an image of an orange, some other neurons will get activated.
- It's similar to what happens inside a human brain. When we see something that we like, some neurons get activated and when we see something that we don't like, some other neurons get activated.
- Now, due to these activations of neurons, we can also call these neurons as activations and as these are the functions We can call them activation functions.



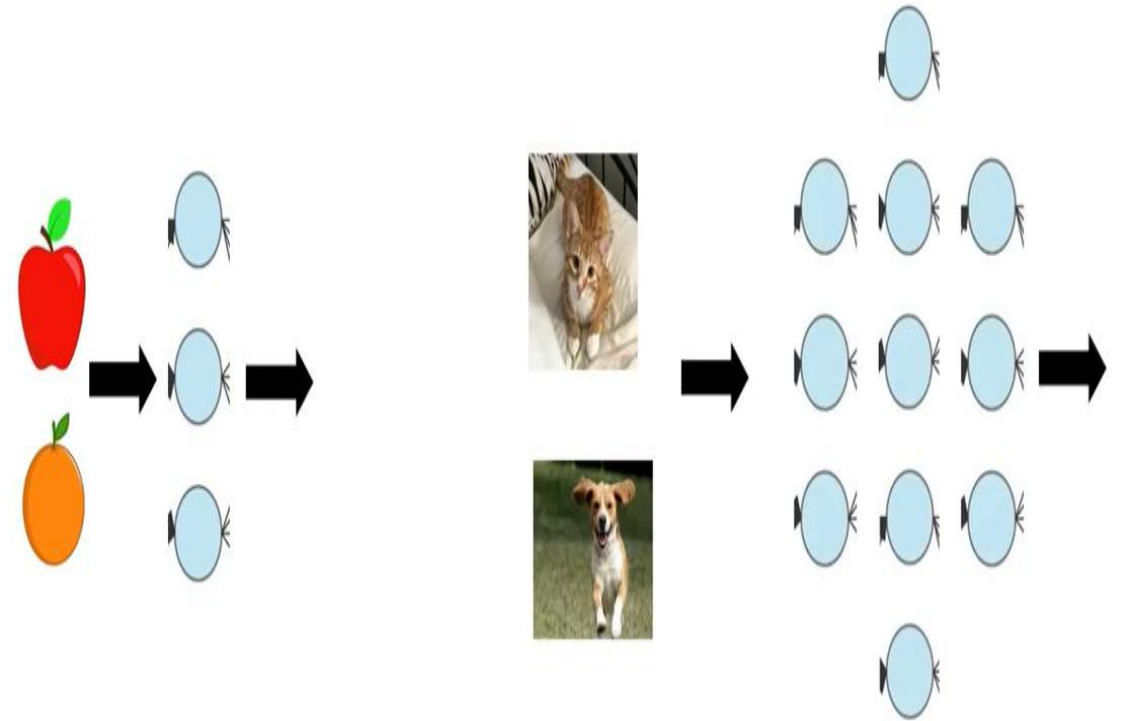
- Now the collection of these neurons forms layers. A neural network is divided into three types of layers. The input layer, the hidden layer and the output layer. The input layer has the neurons which holds the value from the data set. So the number of neurons in the input layer will be equal to the number of features we have in our input data.
- Let's say our image of an apple is of the size 28 pixels into 28 pixels. And the color schema that we are using is RGB. Thus, each pixel will store three values of red, blue and green each.
- Thus, the total number of features in this will be 28 into 28 into 3, which will be 2352. And thus, the total number of neurons in the input layer will be 2352.
- Each neuron holding the color value of every pixel. Now, as our final output can only be an apple or an orange, Thus the output layer will have only one neuron which holds a value between 0 to 1 showing the probability of an image being an apple or an orange. If the value comes out to be greater than 0.5 then we will classify it as an apple. If the value comes out to be less than 0.5 then we will classify it as an orange.



- The hidden layers are responsible for holding the patterns in them.
- It is possible here that our first layer is responsible for finding the shape of the content of the image and as you can see the shape of both apple and orange are slightly different from each other.
- And the second layer might be recognizing the colors in the central region and some neurons will be activated for red color while the other with the orange.
- Now the question is how many neurons should a layer be having and how many layers should we be using? Can we have three or let's say five layers as our hidden layers?
- The answer is it depends on our choice. We can keep as much as we want and as much as it requires for our application.



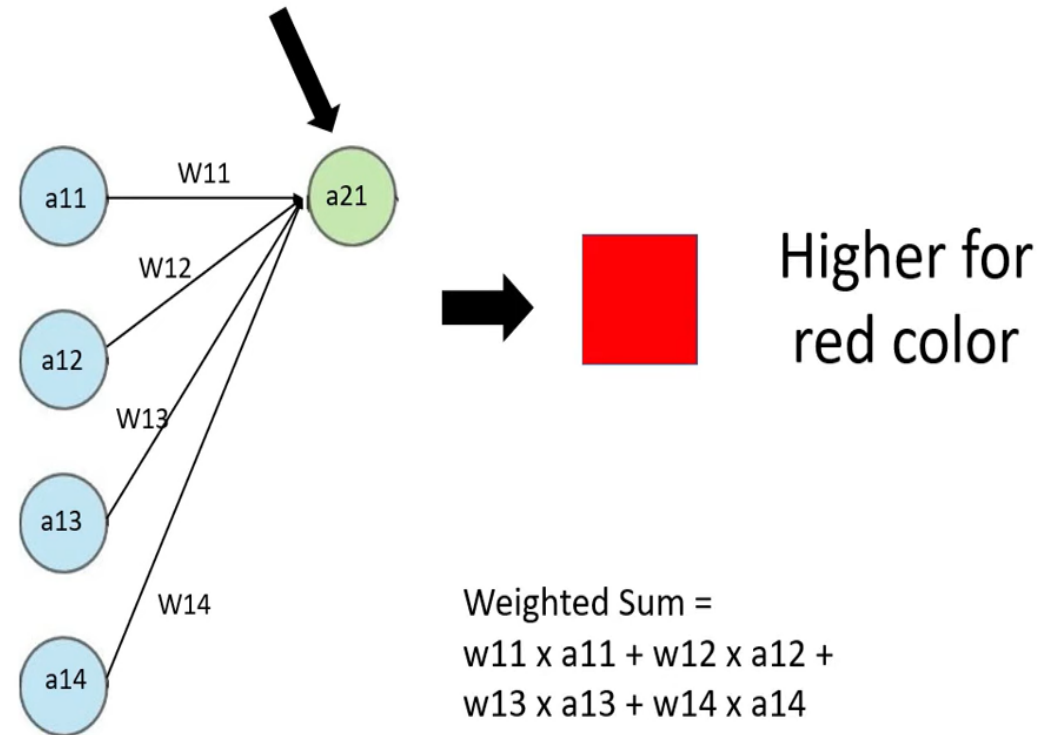
- With the data set of an apple and the oranges, we don't have a lot of information to recognize. Thus, we can keep only one hidden layer with only few neurons or we can also keep two hidden layers.
- But if our job is to classify whether the image is in cat or a dog, then there may be many features involved here and thus we can keep more number of neurons in each hidden layer and more number of hidden layers as well.
- Also, one more thing to note here is that the more the number of neurons and more the number of layers, larger will be our calculation, thus it will take more time to train our model. So, if we can make our model with few neurons and few layers, then we need not use more number of layers.



More Neurons = More Calculations = More time

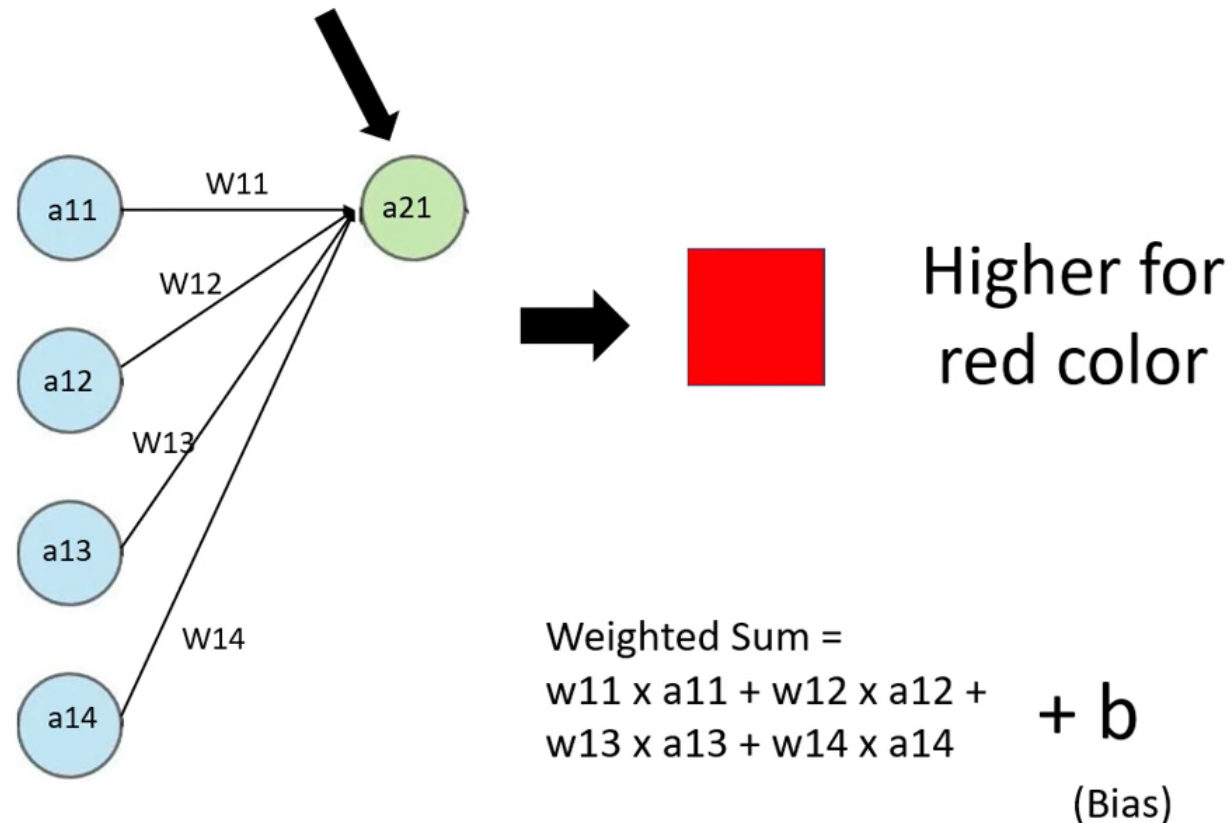
Connections between every pair of neurons

- There is one connection between every two pair of the neurons and we assign a weight value to every pair of the two neurons.
- This weight value is nothing but the parameter that we train. And we call them weights because they determine how much weight should be putting or how much emphasis should be given to a certain region or certain patterns that we are recognizing. This can be done by taking a weighted sum.
- Now, a weighted sum is when we multiply every weight within every value of the neuron and take its sum. Let's say this neuron is responsible for recognizing the color in the central region of the image.
- Now, our weighted sum will be high if the color in the central region of the image is red. Thus, our network will be confident about having the presence of an apple in the image instead of an orange.



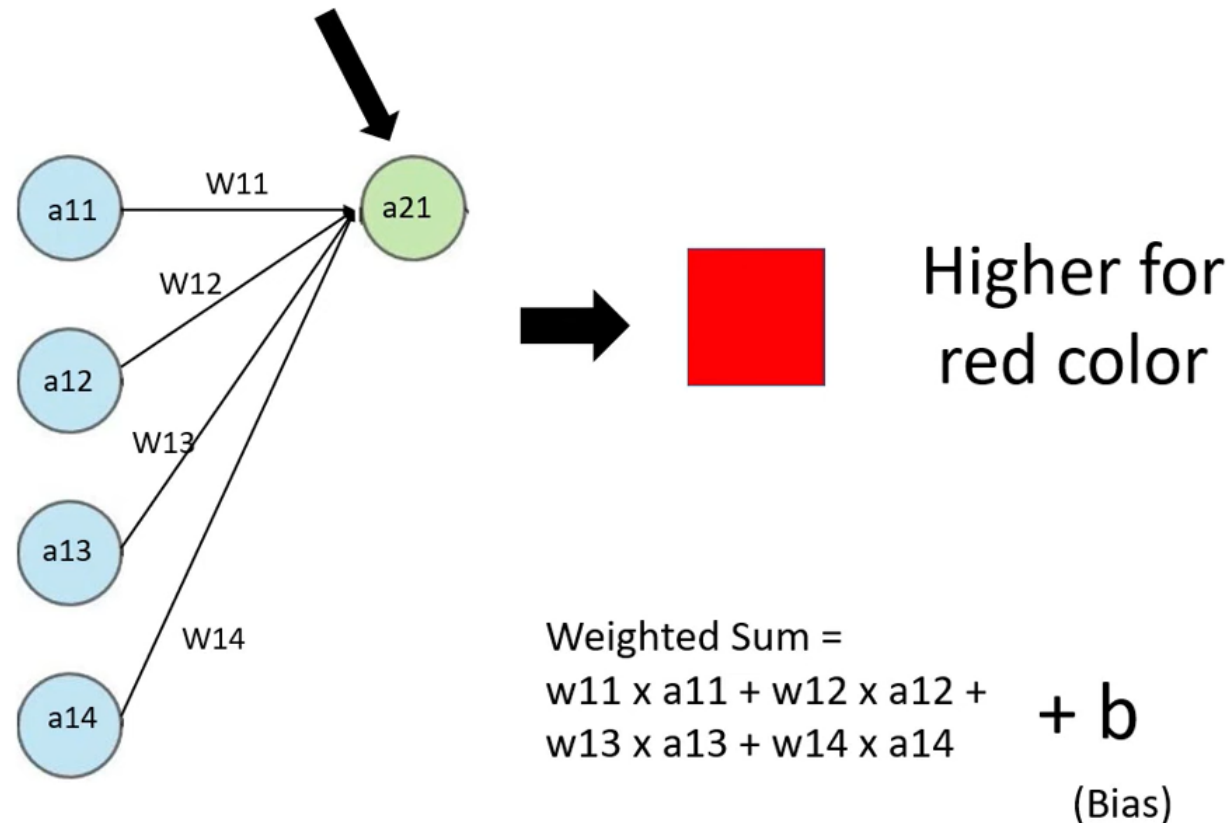
Bias

- This bias value determines how high our weighted sum should be. If the b value, the bias value is negative, then our weighted sum will be less. But if this value is high positive number, then our weighted sum will be high.

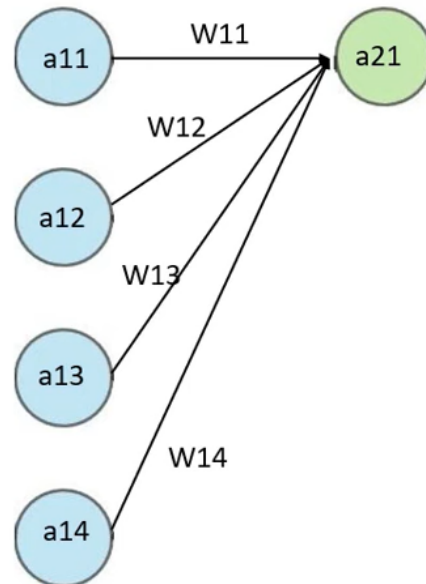


Activation Function

- This weighted sum is then passed to the activation function which gives a proper output value, a single small output value. And this output value only gives the existence of a neuron.



- So, a_{21} will be the output value after passing the weighted sum to the activation function.
- It is also good to notice here that these neurons will again be multiplied with the weights to calculate the weighted sum for the next layer and the process will be repeated until and eventually we generate the final output prediction, which will be a_3 in our case.
- Now, we can ask a question here and that is, we know that we get the value of neurons but taking the weighted sum and passing it in an activation function. But then how do we get the values of these weights? The worst thing to do is to set all these weights manually and that would be a really hectic job and that's not a correct approach.



$$\text{Weighted Sum} = \\ w_{11} \times a_{11} + w_{12} \times a_{12} + \\ w_{13} \times a_{13} + w_{14} \times a_{14} + b$$

$$a_{21} = \text{function} (w_{11} \times a_{11} + \\ w_{12} \times a_{12} + \\ w_{13} \times a_{13} + \\ w_{14} \times a_{14} + \\ b)$$

Weights

- So what we do is that we first initialize random values to these weights and then we train our model and after the model is trained it will automatically change the values of the weight to give the proper output prediction.

$$W = \begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \\ w_{13} & w_{23} \\ w_{14} & w_{24} \end{bmatrix}$$

Random
Weights

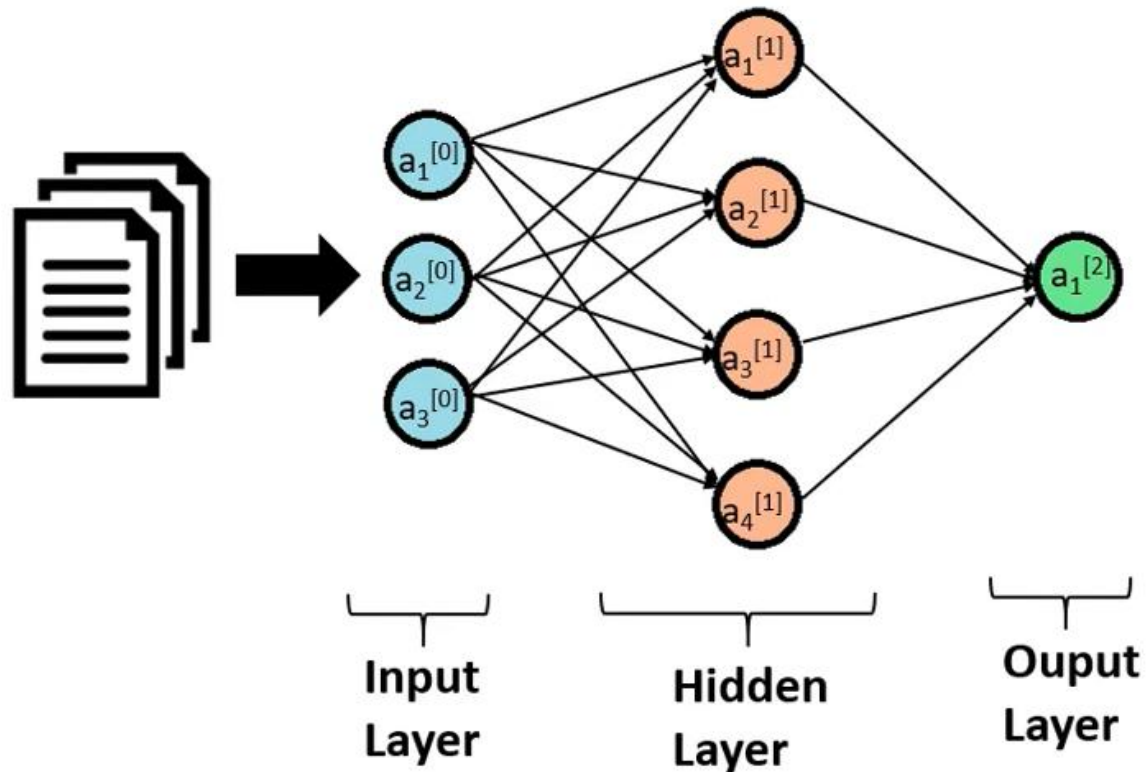


After
Training



Trained
Weights

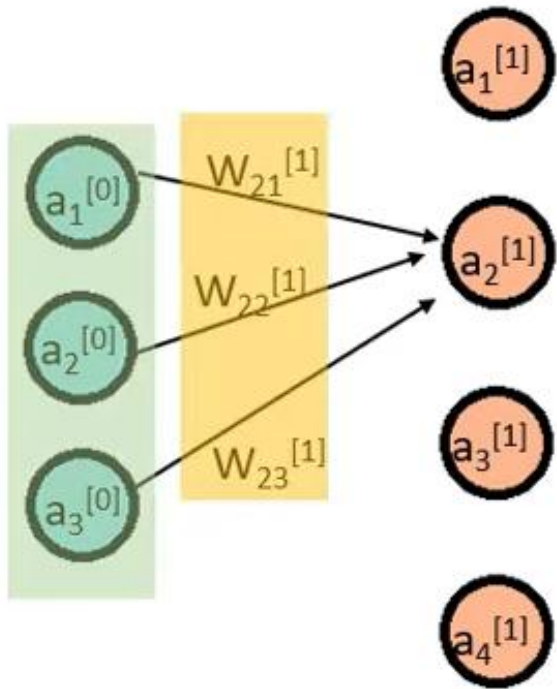
- We passed the input data from the input layer Then we do the weighted sum by multiplying the weights with the neurons and then pass it to the activation function and this defines the neurons in the next layer.



$$a_1^{[1]} = \text{activation_function}(W_{11}^{[1]} * a_1^{[0]} + W_{12}^{[1]} * a_2^{[0]} + W_{13}^{[1]} * a_3^{[0]} + B1)$$

$$a_2^{[1]} = \text{activation_function}(W_{21}^{[1]} * a_1^{[0]} + W_{22}^{[1]} * a_2^{[0]} + W_{23}^{[1]} * a_3^{[0]} + B1)$$

- Here, the superscript square brackets represent the current layer we are in. The input layer as our zeroth layer and the first hidden layer as first layer.
- The subscript represents the position of the weight between the two neurons. For example, the weight is between the first neuron of the first hidden layer and the second neuron of the input layer.



$$a_2^{[1]} = \text{activation_function}($$

$$\begin{aligned} &W_{21}^{[1]} * a_1^{[0]} + \\ &W_{22}^{[1]} * a_2^{[0]} + \\ &W_{23}^{[1]} * a_3^{[0]} + \\ &B1) \end{aligned}$$

- Here, the W is of the shape where NH is the number of neurons in the hidden layer while NX is the number of neurons in the input layer. Now, when we do the matrix multiplication of these two matrices, we get the resultant matrix X . Add a bias and pass it to the activation function to get the values of the neurons in the hidden layer.

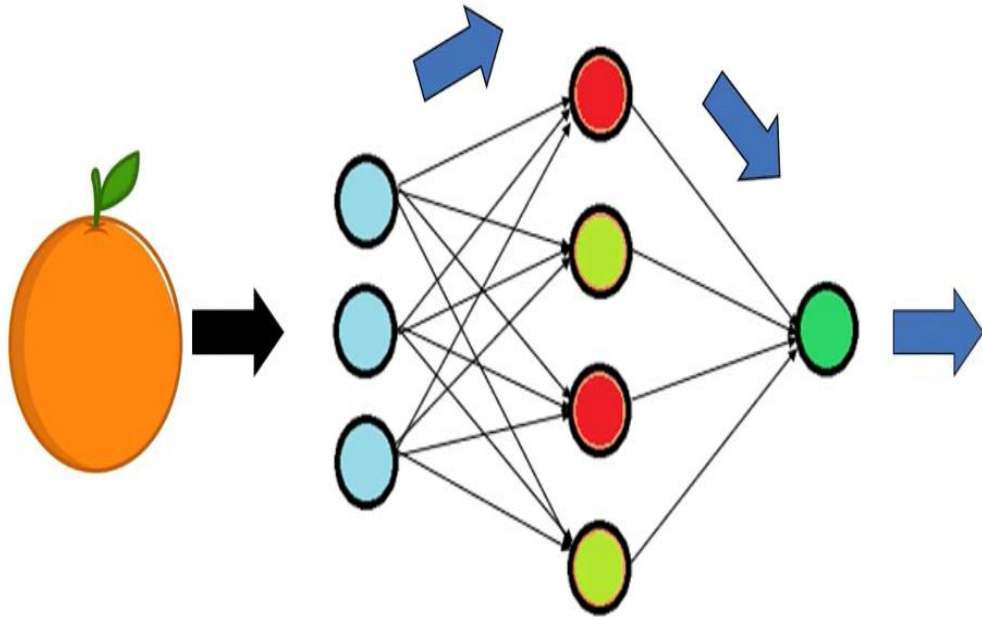
$$\begin{array}{c}
 \mathbf{W} = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{21} & W_{22} & W_{23} \\ W_{31} & W_{32} & W_{33} \\ W_{41} & W_{42} & W_{43} \end{bmatrix} \begin{bmatrix} a_1^{[0]} \\ a_2^{[0]} \\ a_3^{[0]} \end{bmatrix} = \begin{bmatrix} W_{11}^{[1]} * a_1^{[0]} + W_{12}^{[1]} * a_2^{[0]} + W_{13}^{[1]} * a_3^{[0]} \\ W_{21}^{[1]} * a_1^{[0]} + W_{22}^{[1]} * a_2^{[0]} + W_{23}^{[1]} * a_3^{[0]} \\ W_{31}^{[1]} * a_1^{[0]} + W_{32}^{[1]} * a_2^{[0]} + W_{33}^{[1]} * a_3^{[0]} \\ W_{41}^{[1]} * a_1^{[0]} + W_{42}^{[1]} * a_2^{[0]} + W_{43}^{[1]} * a_3^{[0]} \end{bmatrix} \\
 \begin{array}{ccc}
 \text{shape}(n_h, n_x) & \text{shape}(n_x, 1) & \text{shape}(n_h, 1) \\
 = \text{shape}(4, 3) & = \text{shape}(3, 1) & = \text{shape}(4, 1)
 \end{array}
 \end{array}$$

$$f \left(\begin{bmatrix} W_{11}^{[1]} * a_1^{[0]} + W_{12}^{[1]} * a_2^{[0]} + W_{13}^{[1]} * a_3^{[0]} \\ W_{21}^{[1]} * a_1^{[0]} + W_{22}^{[1]} * a_2^{[0]} + W_{23}^{[1]} * a_3^{[0]} \\ W_{31}^{[1]} * a_1^{[0]} + W_{32}^{[1]} * a_2^{[0]} + W_{33}^{[1]} * a_3^{[0]} \\ W_{41}^{[1]} * a_1^{[0]} + W_{42}^{[1]} * a_2^{[0]} + W_{43}^{[1]} * a_3^{[0]} \end{bmatrix} + B1 \right) = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix} = A1$$

$$A1 = f (W1 * A0 + B1) \quad A2 = f (W2 * A1 + B2)$$

*(Here, $W1$ is matrix containing all the weight associated with first layer

- The final output is obtained by propagating the information from the input layer to the final output layer.



Forward Propagation

Forward Propagation

$$Z1 = W1 * A0 + B1$$

$$A1 = f (Z1)$$

$$Z2 = W2 * A1 + B2$$

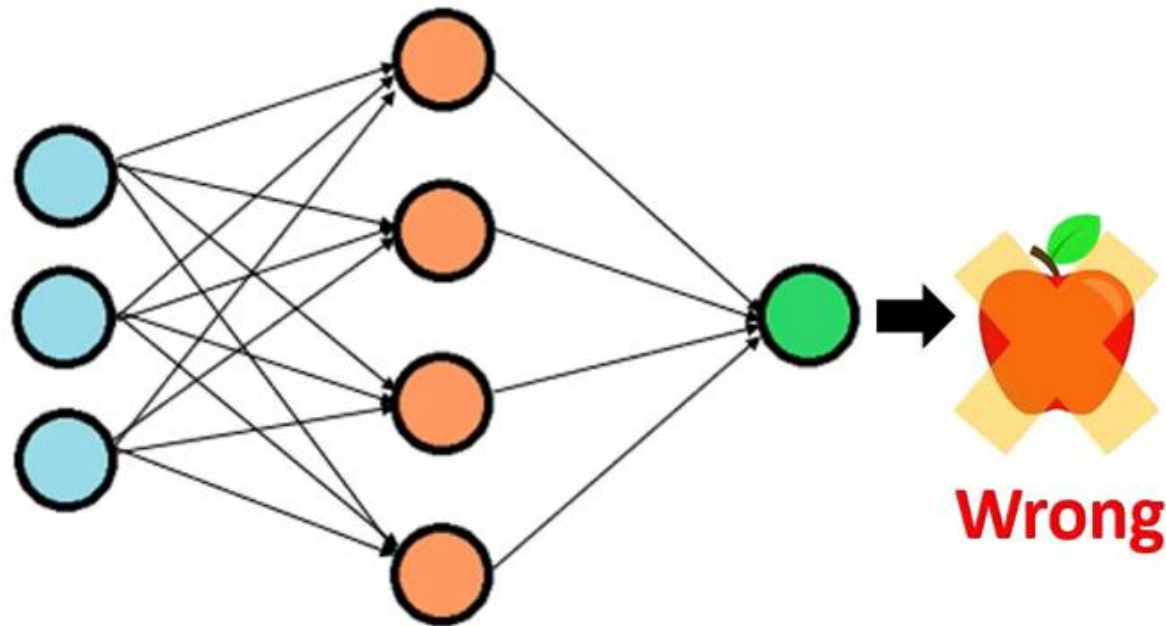
$$A2 = f (Z2)$$

$$Z3 = W3 * A2 + B3$$

$$A3 = f (Z3) = \text{output prediction}$$

- We cannot assign every weight manually as there can be so many weights. So we initialize weights randomly, train the model and after training, values of these weights will be changed in such a way that it can generate a proper output predictions.
- And to do this, first start with the cost function. Cost function in machine learning is the error representation and need to minimize the error.

Random Weights W

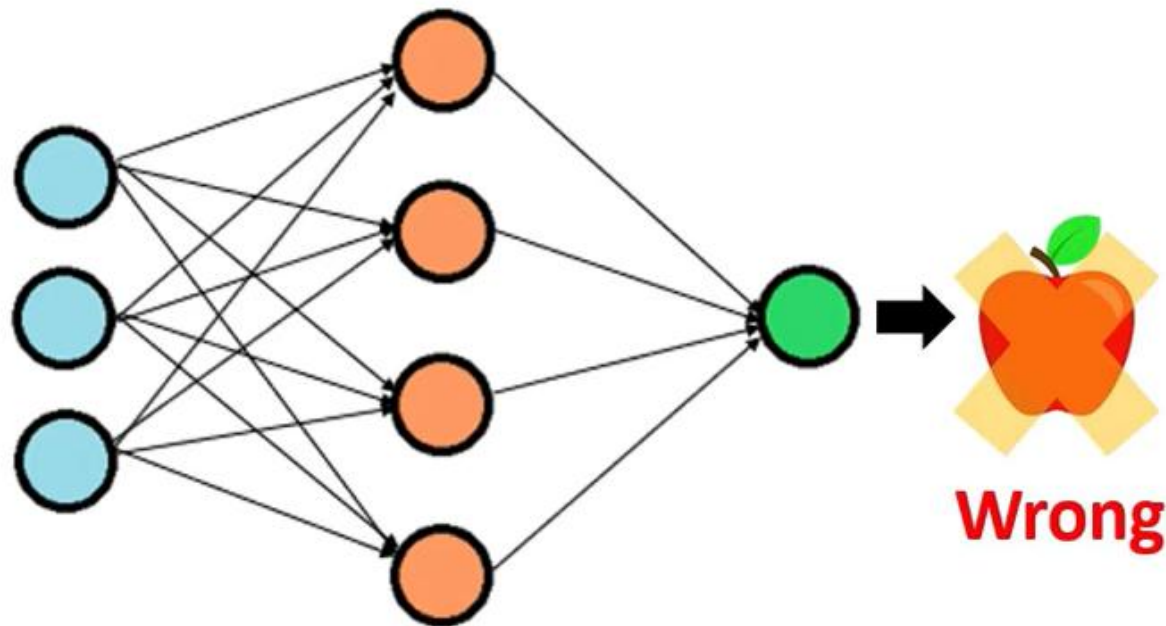


**Cost Function
is high**



- We cannot assign every weight manually as there can be so many weights. So we initialize weights randomly, train the model and after training, values of these weights will be changed in such a way that it can generate a proper output predictions.
- And to do this, first start with the cost function. Cost function in machine learning is the error representation and need to minimize the error.

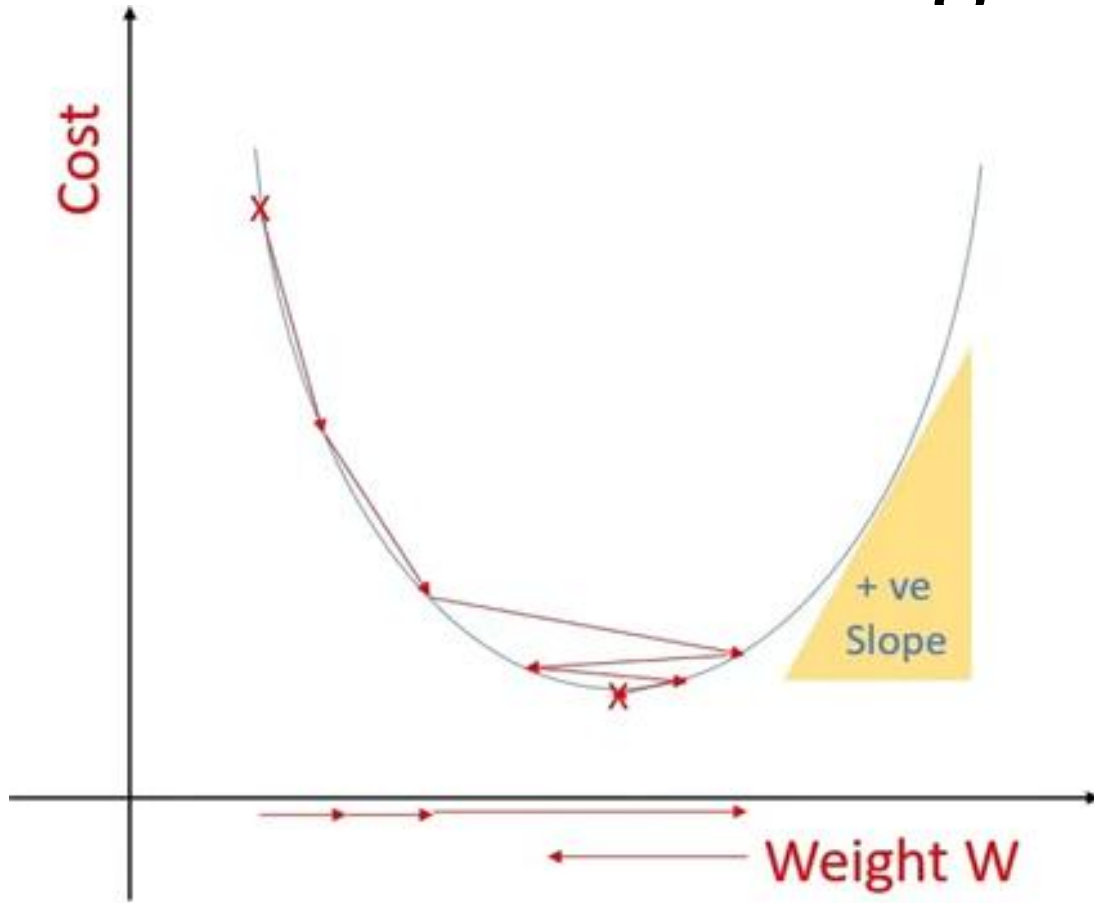
Random Weights W



**Cost Function
is high**



Gradient Descent Algorithm



Repeat {

$$W = W - \text{alpha} * \frac{\partial \text{cost}}{\partial W}$$

}

Gradient Descent

Repeat {

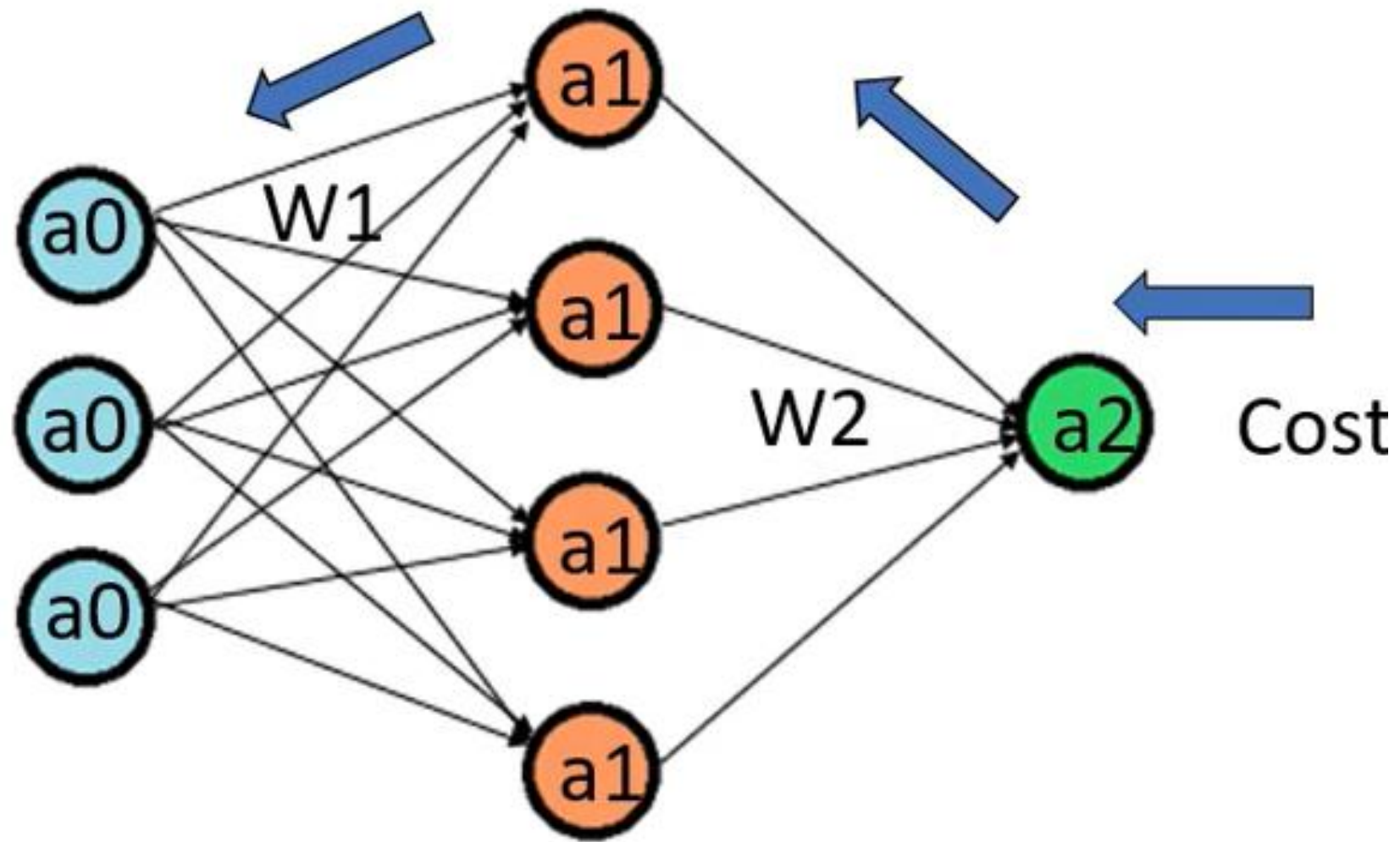
$$W2 = W2 - \text{alpha} * \frac{\partial \text{cost}}{\partial W2}$$

$$B2 = B2 - \text{alpha} * \frac{\partial \text{cost}}{\partial B2}$$

$$W1 = W1 - \text{alpha} * \frac{\partial \text{cost}}{\partial W1}$$

$$B1 = B1 - \text{alpha} * \frac{\partial \text{cost}}{\partial B1}$$

}



How a Neural Network is Trained

Step 1 : Initialize weights randomly Step 4 : Backward Propagation

Step 2 : Forward Propagation

$$Z1 = W1 * A0 + B1$$

$$A1 = f (Z1)$$

$$Z2 = W2 * A1 + B2$$

$$A2 = f (Z2)$$

Step 3 : find value of Cost

$$W2 = W2 - \alpha * \frac{\partial \text{cost}}{\partial W2}$$

$$B2 = B2 - \alpha * \frac{\partial \text{cost}}{\partial B2}$$

$$W1 = W1 - \alpha * \frac{\partial \text{cost}}{\partial W1}$$

$$B1 = B1 - \alpha * \frac{\partial \text{cost}}{\partial B1}$$

Step 5 : Repeat Step 2, 3 and 4 for many times

- No matter how many layers we use, we will be getting the similar kind of equations and thus we will not be able to take the advantage or the benefit of using the multiple layers. And it will become as just using one layer.
- Now without using any activation functions, all these become linear. So we need to use a non-linear activation functions to take the benefit of the different hidden layers. And the function must be non-linear. Also the real world data is non-linear. and thus learning from the complex relationship between the data will be better suited with the help of the non-linear activation functions.

$$Z1 = W1 * A0 + B1$$

~~$$A1 = f(Z1)$$~~

$$Z2 = W2 * A1 + B2$$

~~$$A2 = f(Z2)$$~~

$$Z3 = W3 * A2 + B3$$

~~$$A3 = f(Z3)$$~~

$$A2 = W2 * (W1 * A0 + B1) + B2$$

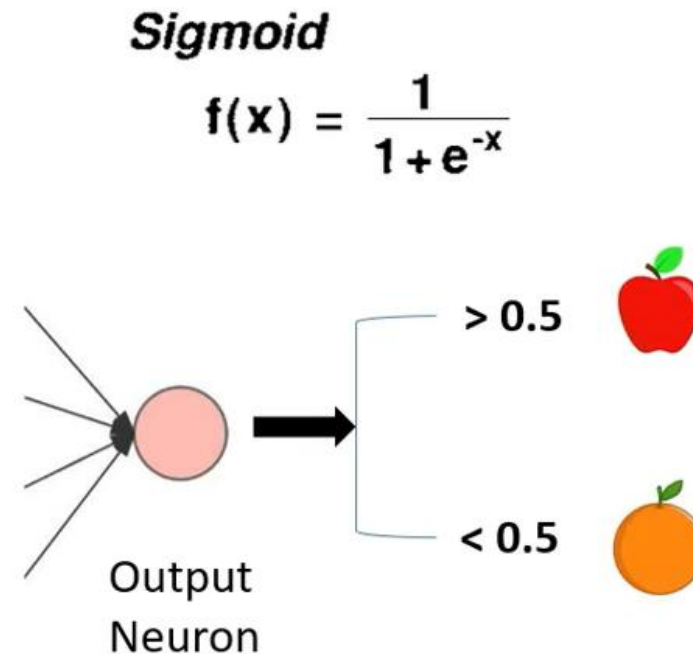
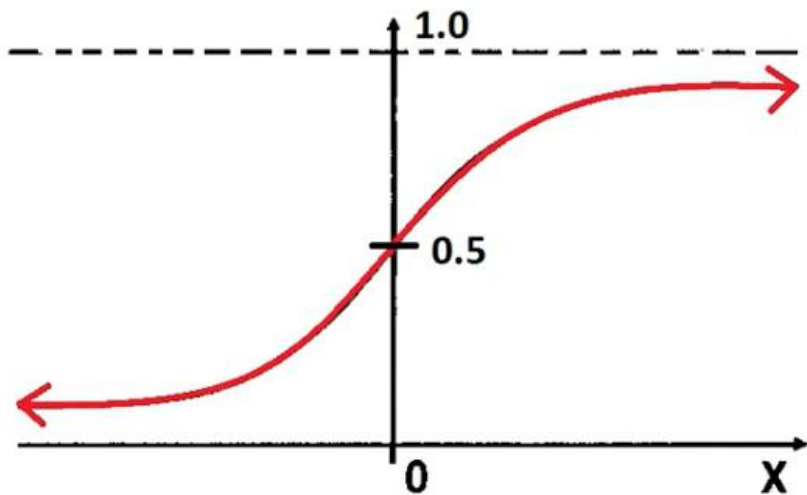
$$A2 = \underbrace{W2 * W1}_{W'} * A0 + \underbrace{W2 * B1 + B2}_{B'}$$

$$A2 = W' * A0 + B'$$

$$A3 = W'' * A0 + B''$$

Activation Function : Sigmoid

- The sigmoid function is an s shape curve and it takes any value between 0 to 1. And for the given 0 input value, it gives 0.5 as the output value. Now, as it only takes the value between 0 to 1, it can be interpreted as a probability if we use this in the output layer.

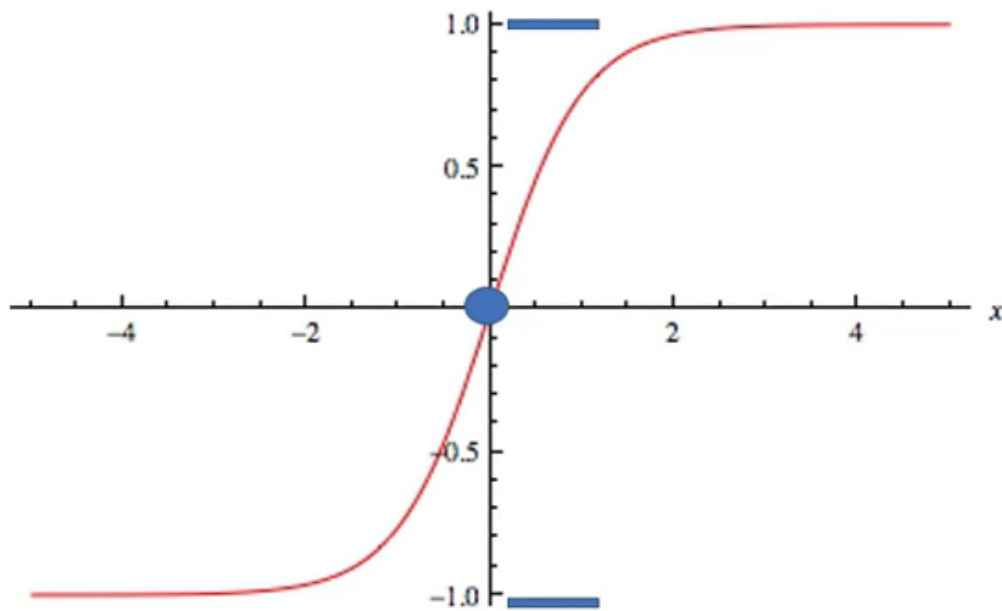


Sigmoid

- Sigmoid function is most commonly used at the output neuron in the binary classification.
- Let's say if we are classifying apples versus oranges and the output value is greater than 0.5, then we can classify it as an apple. And if the output value is less than 0.5 then we can classify it as an orange.

Tanh

- Tanh function or the hyperbolic tangent function is similar to sigmoid function just that it is stretched between minus 1 to 1 instead of 0 to 1.



$$\tanh = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$W = W - \alpha * \frac{\partial \text{cost}}{\partial W}$$

We need to take
derivative of activation
functions

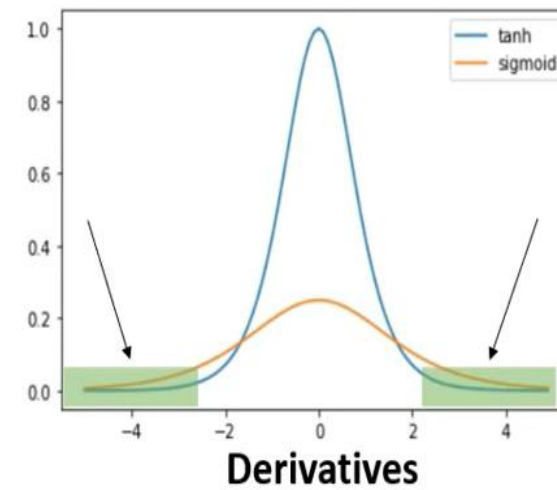
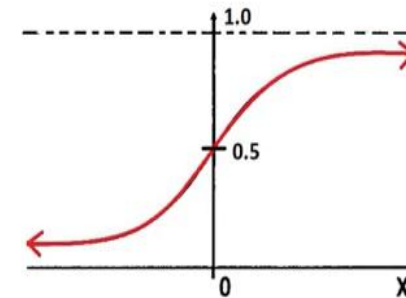
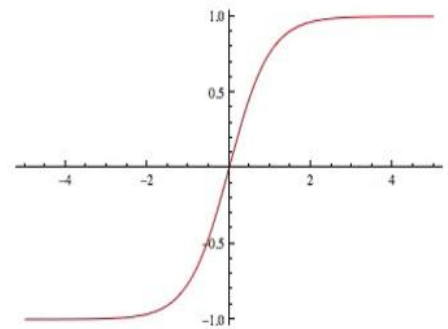
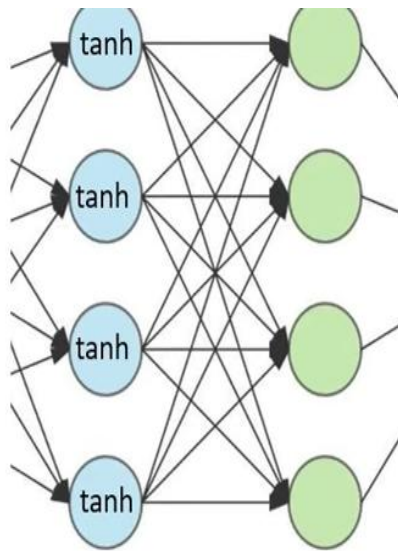
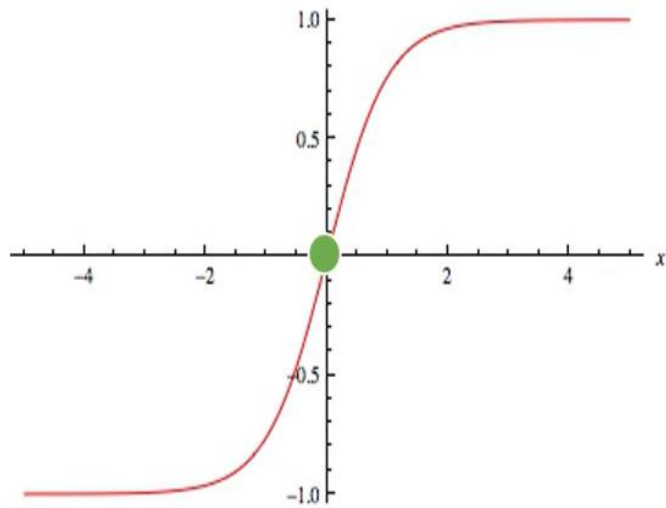
$$\frac{\partial \tanh}{\partial x}$$

Tanh

- It has some advantages over sigmoid function when we use it in the header layer.
- In the back propagation or gradient descent, we need to use the derivative of cost function with respect to w .
- And the cost function depends on the activation function, so we need to take the derivative of the activation functions as well.
- So, if we see the maximum value of the derivative of sigmoid function is only 0.25 while that of tanh is upto 1.
- Thus using tanh activation function we can train our model much faster as the derivative value is higher it will converge much faster.
- Also in tanh function the average of the data will be close to zero and if we use tanh function in the hidden layers then it works much better than the sigmoid function.

Tanh

- It is because of the reason that the output of the hidden layers which is also the input to the next layer, will be having the values whose average will be close to zero. Or we can say that the data is normalized or centered around zero. And when we pass a normalized data to the next layer, it makes the training much more easier.
- Thus we use tanh functions in the hidden layers and not the sigmoid functions.
- Now both these tanh functions and sigmoid functions also has some disadvantage.



Smaller value of derivative leads to very slow learning

Vanishing Gradient Problem

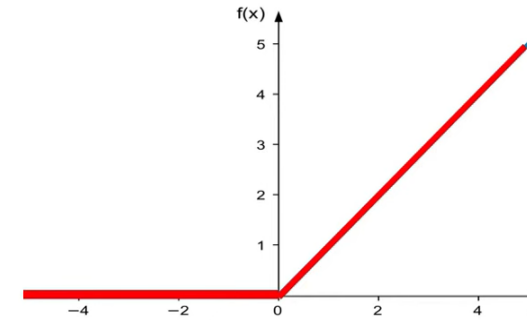
Tanh

- When we take the derivative of both these functions while doing the gradient descent, we see that for a very high value of x or for a very low value of x , the derivative becomes very small.
- Thus, the learning will be very slow. Now, this is called vanishing gradient problem when the learning becomes very slow or the change or the training of the data does not take place effectively.
- This problem can be overcome by using another type of activation function which is called a ReLU function or a rectified linear unit function. Now it is a linear function where we rectify the input which is less than zero.

Tanh

- So for every x less than or equal to zero it gives zero as the output and for every x greater than zero it gives same x as the output. Now, as the values of this derivative is same for all x greater than 0, it overcomes the vanishing gradient problem.
- This function is not entirely linear as it gives 0 as the output for all x less than 0, while it gives linear output for all x greater than 0.
- So it can be also said as a piecewise linear function and this actually makes it even better as it can take the advantage of both linear and non-linear property.
- The linearity of the function can help to overcome the vanishing gradient problem and also makes the training faster .

ReLU (Rectified Linear Unit)



Piece-wise Linear

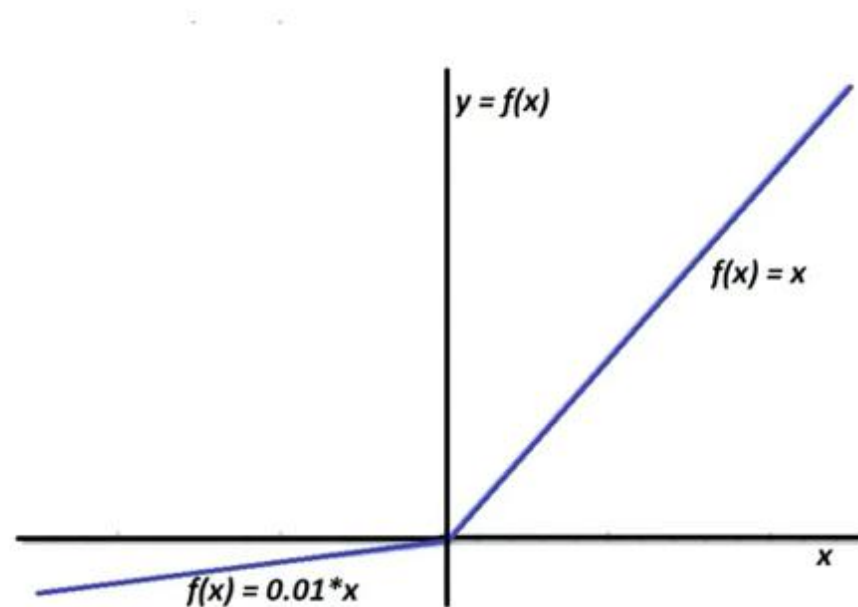
Advantages of both linear and non-linear property

$$f(x) = \max(0, x)$$

Overcome the
Vanishing Gradient
Problem

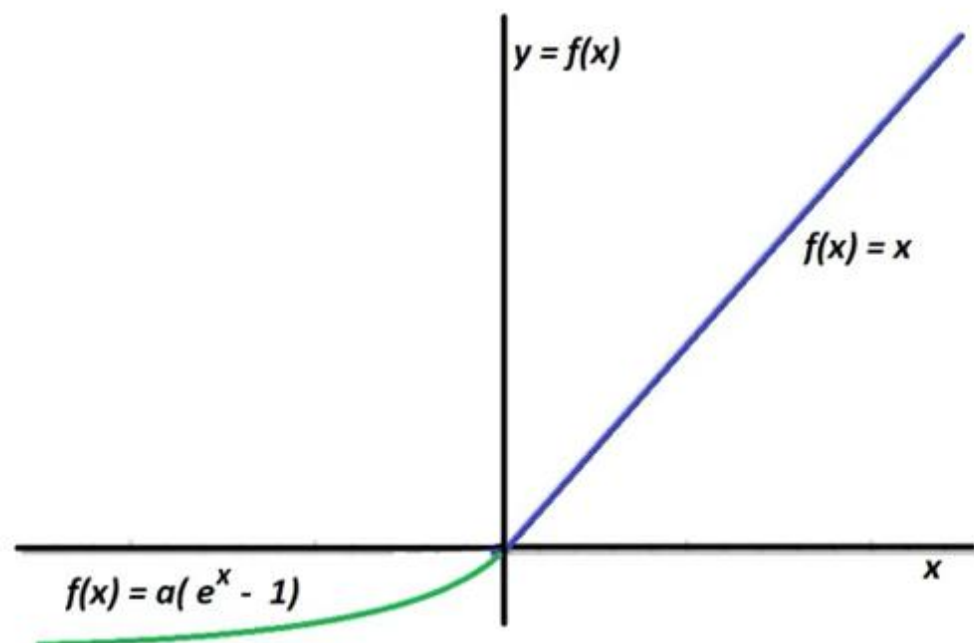
$$\frac{\partial f(x)}{\partial x} = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$$

Variations of ReLU



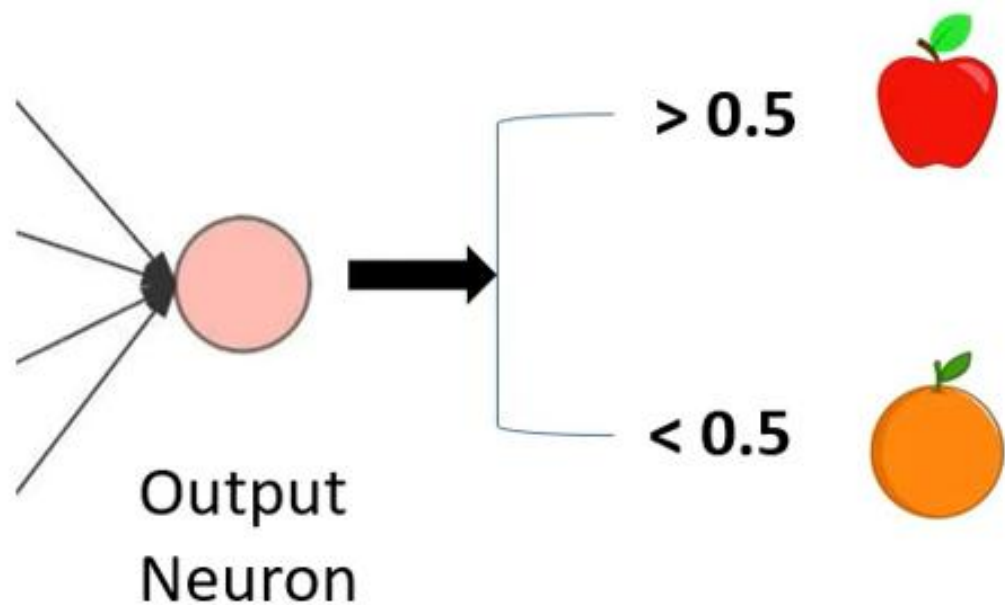
Leaky ReLU

$$f(x) = \max(0.01 * x, x)$$

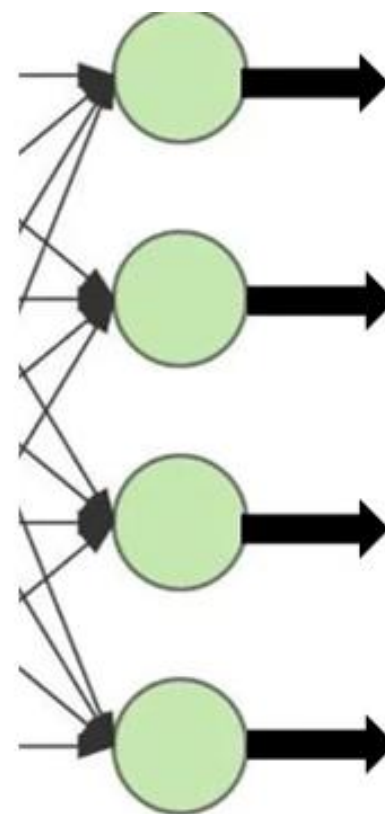


ELU (Exponential Linear Unit)

$$f(x) = \max(\alpha * (e^x - 1), x)$$

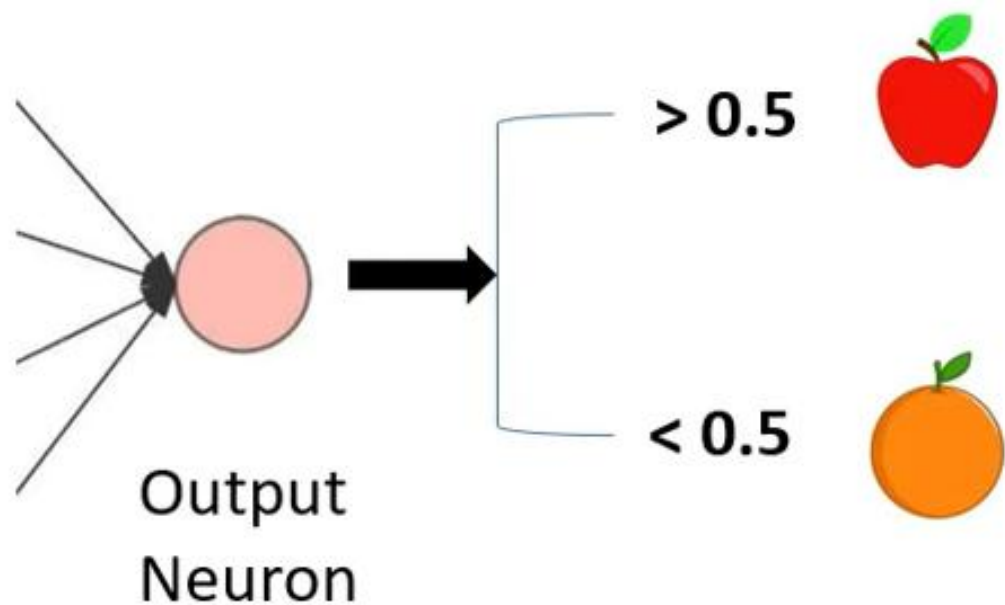


Binary Classification

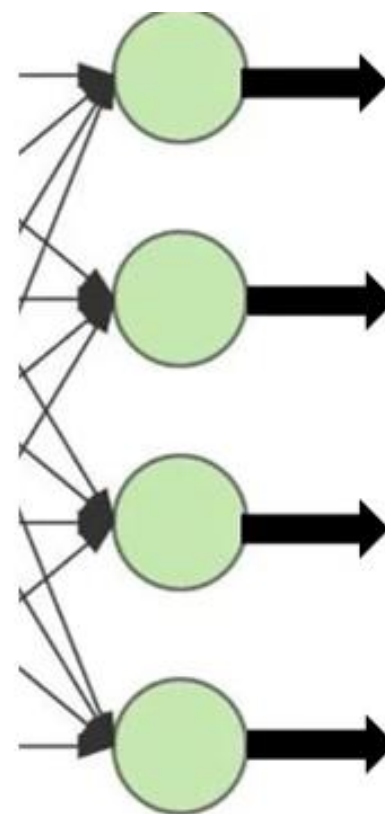


Sigmoid

Multi-Class Classification
??



Binary Classification

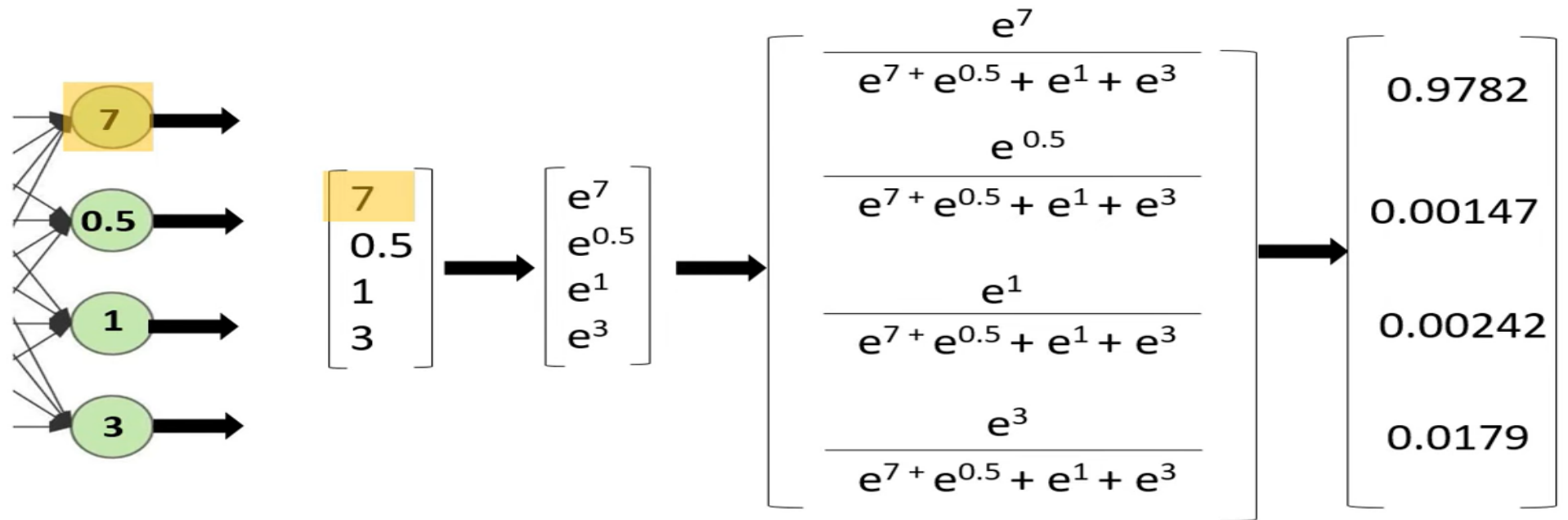


Sigmoid

Multi-Class Classification
??

Output values can be made much better by scaling them into the probabilities. This can be achieved by calculating the exponent of each value in the list and dividing it by the sum of the exponent values. The exponent function makes the larger value much larger and smaller value much smaller.

Softmax Function

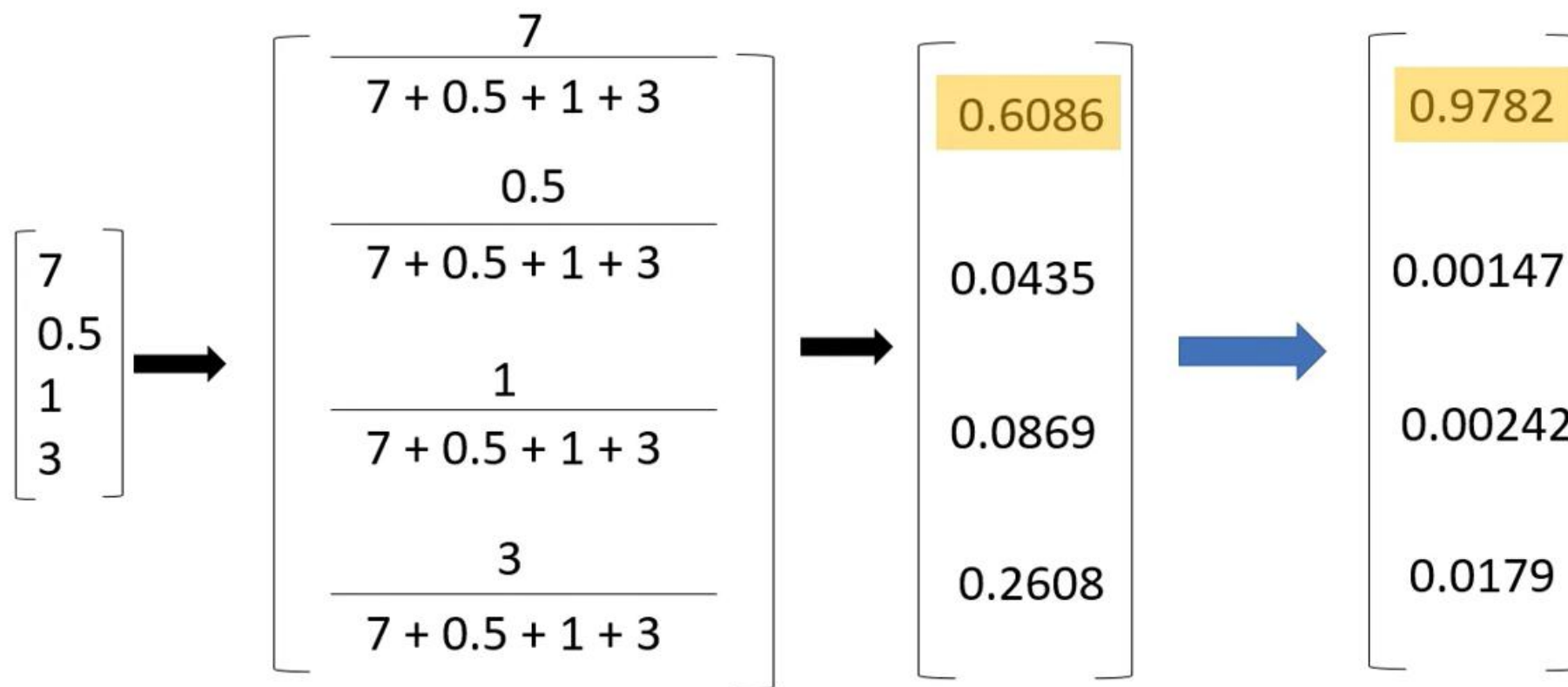


$$0.9782 + 0.00147 + 0.00242 + 0.0179 = 1$$

Thus it enhanced the final output prediction and we can be more sure of our final classification. As you can see with using exponent, the output probability is 0.97 while without exponent is just only 0.6.

Softmax Function

$$\text{Softmax } f_i(x) = \frac{e^{a_i}}{\sum_k e^{a_k}}$$



What if our final output prediction is linear and it can take any value like making the house price prediction. where the price of the house can range from any number between 0 to 1 million or maybe more.

In this case it is not suitable for using any kind of non-linear activation function in the output neuron and just in the output neuron. We will be using the activation functions in the hidden layers but we won't be using any activation functions in the output neuron.

Thus for the output neuron we will be just taking the linear output without activation function for linear regression.



House Price Prediction

*Price of the house can be
anywhere from 0 to \$1 million*



*No Activation
Function in
Output Neuron*

Summary

- For binary classification, we use the sigmoid function in the output neuron because it can be interpreted as the probability. But we don't use sigmoid function in the hidden layers because the tanh and the ReLU function always outperforms the sigmoid function.
- So for hidden layers, we either use tanh function or ReLU function. Now tanh function had one disadvantage which was vanishing gradient problem. and it was that for every very large or very small value of x , the derivative becomes very small and the learning can become very slow. And this can lead our model to not train effectively.
- So it can be overcome with the help of a ReLU function which was a piecewise linear function and it has the advantage of both the linear and the non-linear property. There are other variations of ReLU functions also available such as leaky ReLU and the exponential linear unit.
- And while solving the multi-class classification problem, we use softmax function in the output layers.
- And when solving the linear regression problem, we skip using any kind of activation function or non-linear activation function and directly pass z as our output.

Cost function

- Cost function in machine learning is the error representation. It shows how our model is predicting compared to the actual values or actual output labels. Lesser the cost function, more the accuracy of our model and higher the cost function, lesser the accuracy of our model.

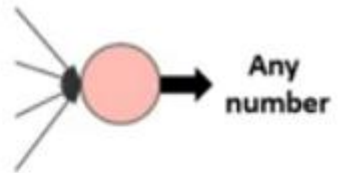
For Regression Problem

$$\text{error} = |a_i - y_i|$$

$$\text{cost} = \frac{1}{m} \sum_{i=1}^m |a_i - y_i| \Leftarrow \text{Mean Absolute Error}$$

$$\text{cost} = \frac{1}{2m} \sum_{i=1}^m |a_i - y_i|^2 \Leftarrow \text{Mean Squared Error}$$

$$\text{Loss} = |a_i - y_i| \quad i^{\text{th}} \text{ observation}$$



House Price
Prediction

Cost function

- Cost function in machine learning is the error representation. It shows how our model is predicting compared to the actual values or actual output labels. Lesser the cost function, more the accuracy of our model and higher the cost function, lesser the accuracy of our model.

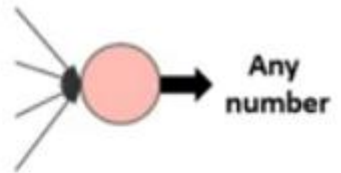
For Regression Problem

$$\text{error} = |a_i - y_i|$$

$$\text{cost} = \frac{1}{m} \sum_{i=1}^m |a_i - y_i| \Leftarrow \text{Mean Absolute Error}$$

$$\text{cost} = \frac{1}{2m} \sum_{i=1}^m |a_i - y_i|^2 \Leftarrow \text{Mean Squared Error}$$

$$\text{Loss} = |a_i - y_i| \quad i^{\text{th}} \text{ observation}$$



House Price
Prediction

Cost function

AI approaches the value closer to Y_i , we get less error and if the AI is far from the Y_i , we get the more error. Thus this term can be represented as the error for one observation

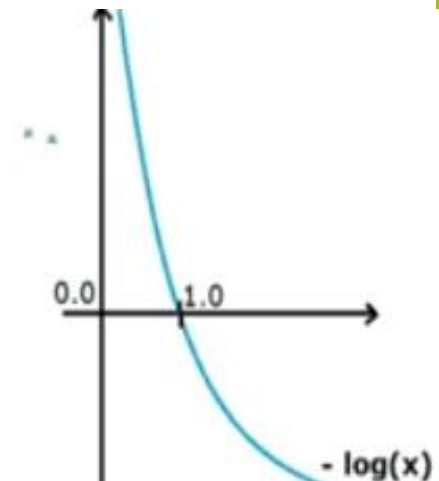
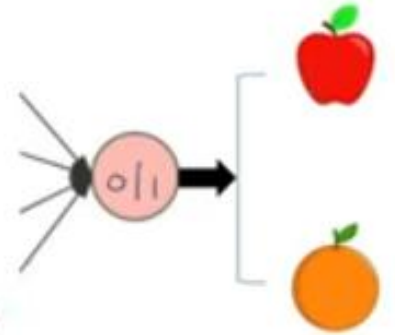
For Binary Classification

$$\text{error}_i = -[y_i \log(a_i) + (1-y_i) \log(1-a_i)]$$

$$\text{cost} = \frac{1}{m} \sum_i [y_i \log(a_i) + (1-y_i) \log(1-a_i)]$$

$$y_i = 0$$

$$\text{error}_i = -(1-y_i) \log(1-a_i) = -\log(1-a_i)$$



Cost function

For Multi-Class Classification

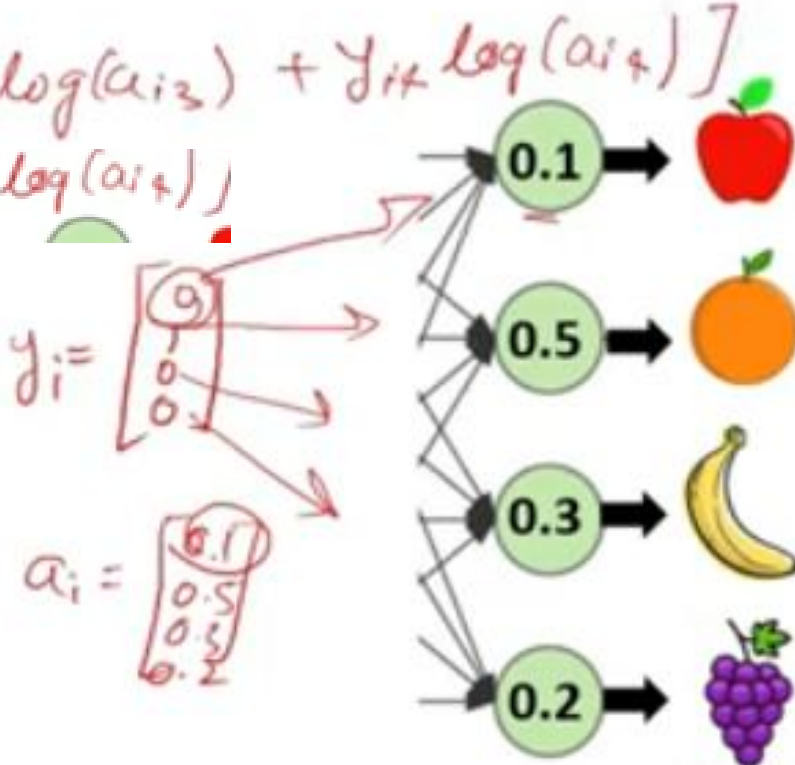
$$\text{error}_i = -[y_{i1} \log(a_{i1}) + y_{i2} \log(a_{i2}) + y_{i3} \log(a_{i3}) + y_{i4} \log(a_{i4})]$$
$$\text{cost} = \frac{1}{m} \sum_i [y_{i1} \log(a_{i1}) + y_{i2} \log(a_{i2}) + y_{i3} \log(a_{i3}) + y_{i4} \log(a_{i4})]$$

$\text{error}_i = -\log(0.5)$

$n = \text{total neurons in output layer}$
 $m = \text{total number of observations}$

$y_i = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$

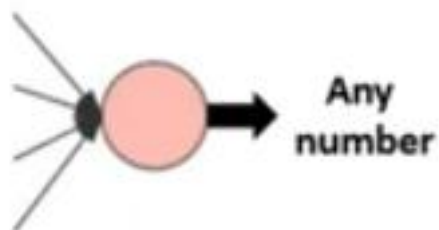
$a_i = \begin{bmatrix} 0.1 \\ 0.5 \\ 0.3 \\ 0.2 \end{bmatrix}$



$$\text{cost} = - \sum_{j=0}^M \sum_{i=0}^N (y_{ij} * \log(a_{ij}))$$

Categorical Cross Entropy

1.) Regression

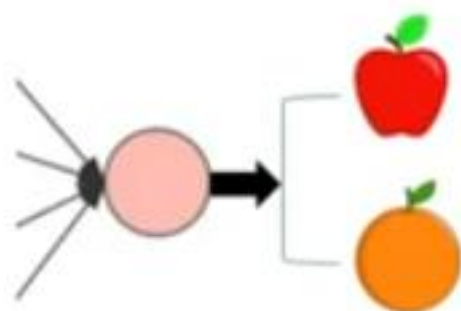


$$\text{Cost} = \frac{1}{m} \sum_{i=1}^m |a_i - y_i|$$

or

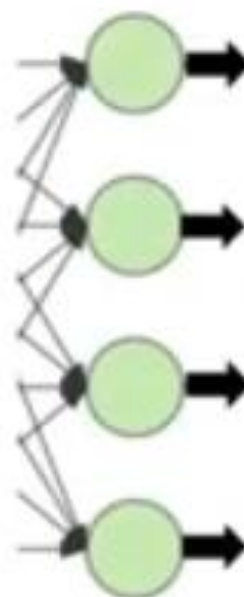
$$\text{Cost} = \frac{1}{2m} \sum_{i=1}^m [a_i - y_i]^2$$

2.) Binary Classification



$$\text{Cost} = -\frac{1}{m} \sum_{i=1}^m [y_i * \log(a_i) + (1 - y_i) * \log(1 - a_i)]$$

3.) Multi-class classification



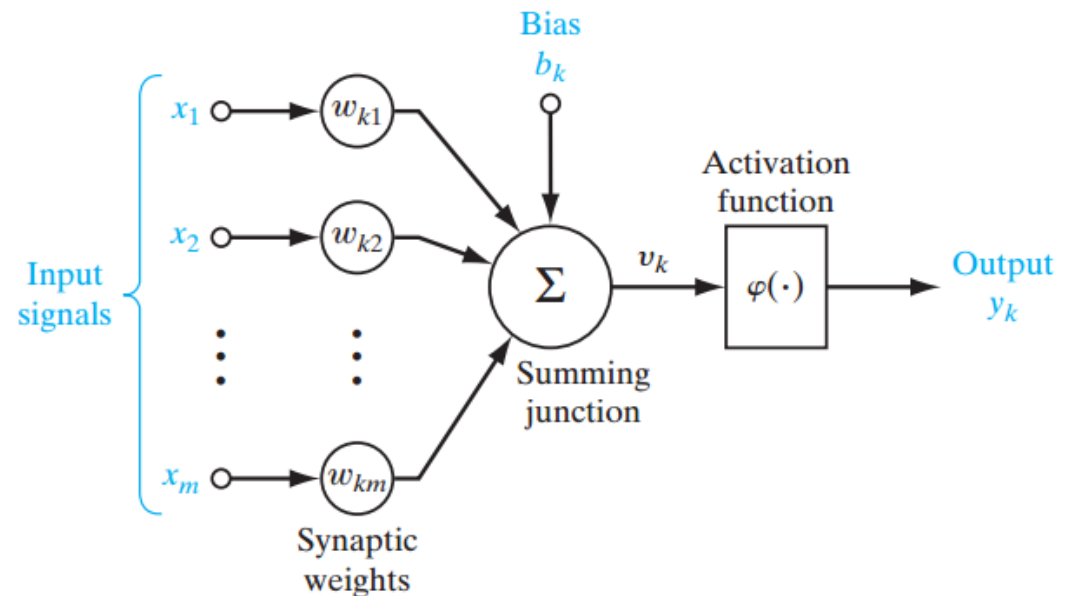
$$\text{cost} = - \sum_{j=0}^M \sum_{i=0}^N (y_{ij} * \log(a_{ij}))$$

Single Layer Perceptron (SLP)

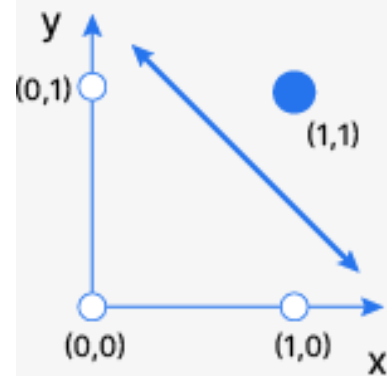
- The simplest type of perceptron has a single layer of weights connecting the inputs and output.
- In this way, it can be considered the simplest kind of feed-forward network.
- In a feed forward network, the information always moves in one direction; it never goes backwards.

$$u_k = \sum_{j=1}^m w_{kj} x_j$$

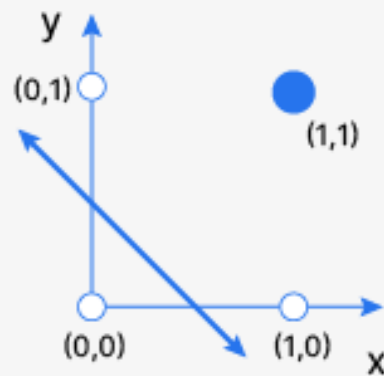
$$v_k = u_k + b_k$$



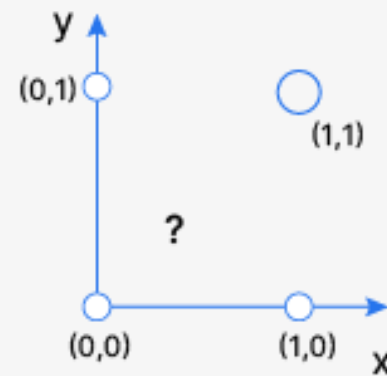
Linear separability



x and y



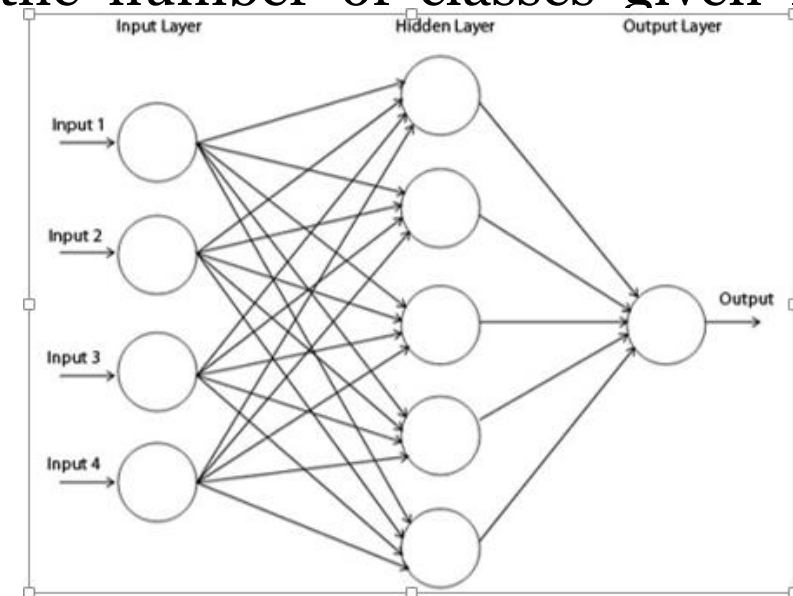
x and y



x and y

Multilayer Perceptron (MLP)

- A multilayer perceptron (MLP) has one or more hidden layers along with the input and output layers, each layer contains several neurons that interconnect with each other by weight links.
- The number of neurons in the input layer will be the number of attributes in the dataset, neurons in the output layer will be the number of classes given in the dataset.



Learning rate

- Learning rate controls the change in the weight from one iteration to another.
- As a general rule, smaller learning rates are considered as stable but cause slower learning.
- On the other hand higher learning rates can be unstable causing oscillations and numerical errors but speed up the learning.

$$\begin{pmatrix} \text{Weight} \\ \text{correction} \\ \Delta w_{ji}(n) \end{pmatrix} = \begin{pmatrix} \text{learning -} \\ \text{rate parameter} \\ \eta \end{pmatrix} * \begin{pmatrix} \text{local} \\ \text{gradient} \\ \delta_j(n) \end{pmatrix} * \begin{pmatrix} \text{input signal} \\ \text{of neuron } j, \\ y_i(n) \end{pmatrix}$$

Momentum

- Momentum provides inertia to escape local minima; the idea is to simply add a certain fraction of the previous weight update to the current one, helping to avoid becoming stuck in local minima.

$$\Delta w_{ij}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_i(n).$$

where α is the momentum.

1. Trainable Parameters Between the Input layer and First Hidden Layer:

$$\text{weights} = 3 * 4 = 12$$

$$\text{biases} = 4 \text{ (4 nodes in 1st hidden layer)}$$

$$\text{Trainable Parameters} = \text{weights} + \text{biases}$$

$$= 12 + 4 = 16$$

2. Trainable Parameters Between First Hidden layer and Second Hidden I

$$\text{weights} = 4 * 2 = 8$$

$$\text{biases} = 2 \text{ (2 nodes in 1st hidden layer)}$$

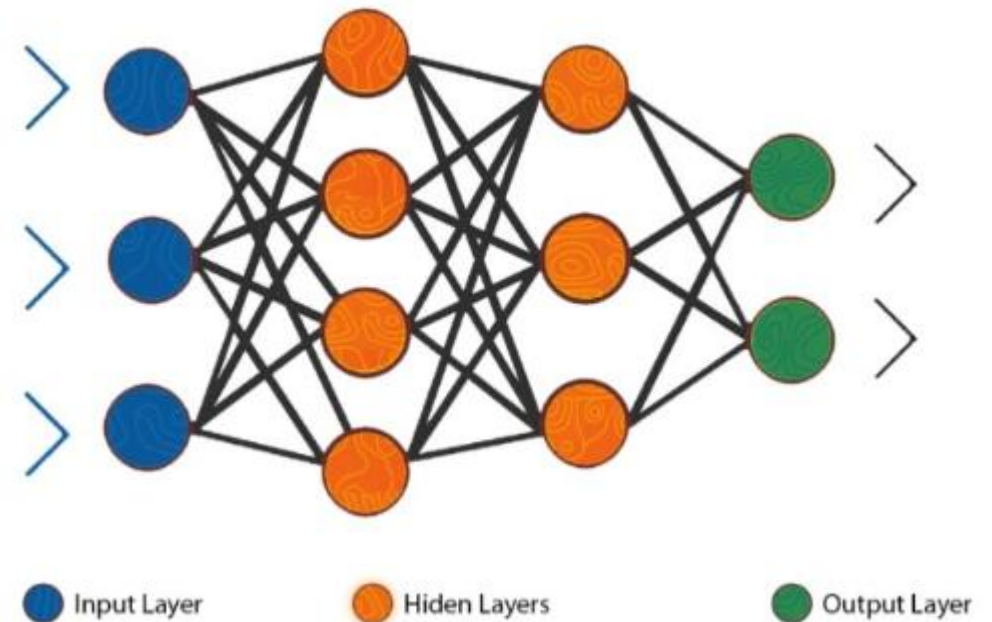
$$\text{Trainable Parameters} = \text{weights} + \text{biases} = 8 + 2 = 10$$

3. Trainable Parameters Between Second Hidden layer and Output Layer:

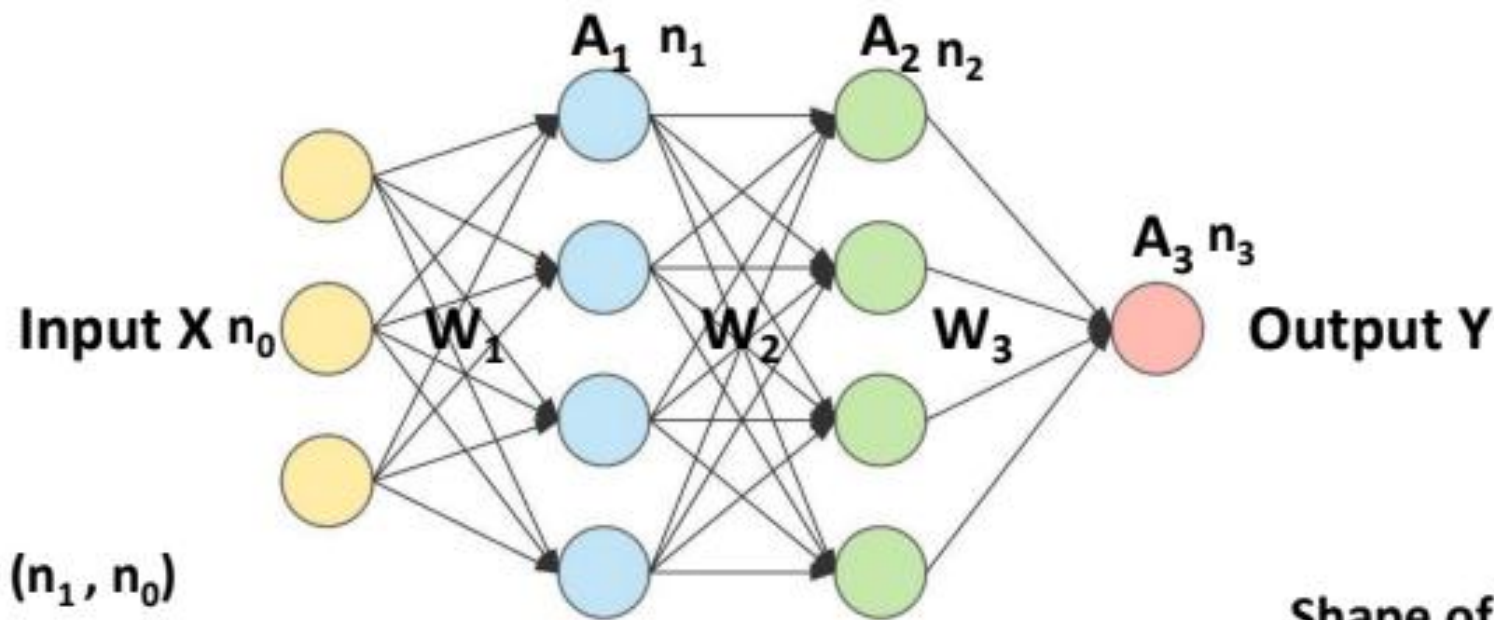
$$\text{weights} = 2 * 2 = 4$$

$$\text{biases} = 2 \text{ (2 nodes in 1st hidden layer)}$$

$$\text{Trainable Parameters} = \text{weights} + \text{biases} = 4 + 2 = 6$$



m = total number of observations



Shape of $W_1 = (n_1, n_0)$

Shape of $W_2 = (n_2, n_1)$

Shape of $W_3 = (n_3, n_2)$

Shape of $B_1 = (n_1, 1)$

Shape of $B_2 = (n_2, 1)$

Shape of $B_3 = (n_3, 1)$

$$A_1 = f_1(Z_1) \quad A_2 = f_2(Z_2) \quad A_3 = \text{Sigmoid}(Z_3)$$

$$Z_k = W_k A_{k-1} + B_k$$

$$\text{Cost} = -\frac{1}{m} \sum_{i=1}^m [y_i * \log(a_i) + (1 - y_i) * \log(1 - a_i)]$$

$$\text{Loss}_i = -[y_i * \log(a_i) + (1 - y_i) * \log(1 - a_i)]$$

Shape of $X = (n_0, m)$

Shape of $A_1 = (n_1, m)$

Shape of $A_2 = (n_2, m)$

Shape of $A_3 = (n_3, m)$

Shape of $Y = (n_3, m)$

$$\text{Loss}_i = - [y_i * \log(a_i) + (1 - y_i) * \log(1 - a_i)] \quad a_3 = \text{Sigmoid}(z_3) \quad z_3 = w_3 A_2 + B_3$$

$$\text{Sigmoid} \\ f(x) = \frac{1}{1 + e^{-x}}$$

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial a_3} * \frac{\partial a_3}{\partial z_3} * \frac{\partial z_3}{\partial w_3}$$

$$\begin{aligned} \frac{\partial L}{\partial a_3} &= \frac{\partial}{\partial a_3} [-y \cdot \log(a_3) - (1-y) \cdot \log(1-a_3)] \\ &= -\frac{y}{a_3} + \frac{(1-y)}{(1-a_3)} \\ &= \frac{-y + y a_3 + a_3 - y a_3}{a_3 \cdot (1-a_3)} \end{aligned}$$

$$\frac{\partial L}{\partial a_3} = \frac{a_3 - y}{a_3 \cdot (1-a_3)}$$

$$\begin{aligned} \frac{\partial a_3}{\partial z_3} &= \frac{\partial}{\partial z_3} \left(\frac{1}{1 + e^{-z_3}} \right) = \frac{\partial}{\partial z_3} [(1 + e^{-z_3})^{-1}] \\ &= (-1) \cdot (1 + e^{-z_3})^{-2} \cdot \frac{\partial}{\partial z_3} (1 + e^{-z_3}) \\ &= \frac{e^{-z_3}}{(1 + e^{-z_3})^2} \end{aligned}$$

$$= \frac{a_3^2 \cdot (1-a_3)}{a_3} \quad \left[\because a_3 = \frac{1}{1 + e^{-z_3}} \right. \\ \left. \Rightarrow e^{-z_3} = \frac{1-a_3}{a_3} \right]$$

$$\frac{\partial a_3}{\partial z_3} = a_3 \cdot (1-a_3)$$

$$\begin{aligned} \frac{\partial z_3}{\partial w_3} &= \frac{\partial}{\partial w_3} [w_3 a_2 + B_3] \\ &= a_2 \end{aligned}$$

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \times \frac{\partial z_3}{\partial w_3}$$

$$= \frac{(a_3 - y)}{a_3 \cdot (1 - a_3)} \cdot a_3 \cdot (1 - a_3) \cdot a_2$$

$$= (a_3 - y) \cdot a_2 \quad \leftarrow \text{for 1 observation}$$

$$\frac{\partial \text{cost}}{\partial w_3} = \frac{1}{m} (A_3 - Y) \cdot A_2^T$$

$\leftarrow$ For all m observations

$$= \frac{1}{m} dz_3 \cdot A_2^T$$

$$\text{where } \frac{\partial \text{cost}}{\partial z_3} = dz_3 = A_3 - Y$$

$$\text{Similarly, } \frac{\partial L}{\partial b_3} = \frac{\partial L}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \times \frac{\partial z_3}{\partial b_3} = (a_3 - y) \cdot (1)$$

$$\frac{\partial \text{cost}}{\partial b_3} = \frac{1}{m} \text{sum}(dz_3, 1)$$

$\uparrow$ Represent column wise summation

Now, For $\partial \text{cost} / \partial w_2$, $\partial \text{cost} / \partial b_2$

$$\begin{aligned} \frac{\partial L}{\partial w_2} &= \frac{\partial L}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \times \frac{\partial z_3}{\partial a_2} \times \frac{\partial a_2}{\partial z_2} \times \frac{\partial z_2}{\partial w_2} \\ &= dz_3 \cdot w_3 \cdot f_2'(z_2) \cdot a_1 \\ \frac{\partial \text{cost}}{\partial w_2} &= \frac{1}{m} [w_3^T \cdot dz_3 * f_2'(z_2)] \cdot A_1^T \\ &= \frac{1}{m} dz_2 \cdot A_1^T \end{aligned}$$

$$[\therefore \frac{\partial L}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} = \frac{\partial L}{\partial z_3} = dz_3]$$

$$\frac{\partial z_3}{\partial a_2} = \frac{\partial (w_3 \cdot a_2 + b_3)}{\partial a_2} = w_3, \quad \frac{\partial a_2}{\partial z_2} = f_2'(z_2)$$

$$\frac{\partial z_2}{\partial w_2} = \frac{\partial (w_2 \cdot a_1 + b_2)}{\partial w_2} = a_1$$

$$[\therefore dz_2 = \frac{\partial \text{cost}}{\partial z_2} = w_3^T \cdot dz_3 * f_2'(z_2)]$$

Similarly, $\frac{\partial \text{cost}}{\partial b_2} = \frac{1}{m} \text{Sum}(dz_2, 1)$

$$[\therefore \text{column wise sum of } dz_2]$$

Similarly, for $\partial \text{cost} / \partial w_1$

$$\begin{aligned} \frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial a_3} \times \frac{\partial a_3}{\partial z_3} \times \frac{\partial z_3}{\partial a_2} \times \frac{\partial a_2}{\partial z_2} \times \frac{\partial z_2}{\partial a_1} \times \frac{\partial a_1}{\partial z_1} \times \frac{\partial z_1}{\partial w_1} \\ &= dz_2 \cdot w_2 \cdot f'_1(z_1) \cdot x \end{aligned}$$

$$\begin{aligned} \frac{\partial \text{cost}}{\partial w_1} &= \frac{1}{m} [w_2^T \cdot dz_2 * f'_1(z_1)] \cdot x^T \\ &= \frac{1}{m} dz_1 \cdot x^T \end{aligned}$$

Similarly,

$$\frac{\partial \text{cost}}{\partial b} = \frac{1}{m} \text{Sum}(dz_1, 1)$$

Complete Back Propagation

$$dZ_3 = (A_3 - Y)$$

$$dW_3 = \frac{1}{m} \cdot dZ_3 \cdot A_2^T$$

$$dB_3 = \frac{1}{m} \cdot \text{sum}(dZ_3, 1)$$

$$W_3 = W_3 - \text{alpha} * dW_3$$

$$B_3 = B_3 - \text{alpha} * dB_3$$

$$dZ_2 = W_3^T \cdot dZ_3 * f_2'(Z_2)$$

$$W_2 = W_2 - \text{alpha} * dW_2$$

$$dW_2 = \frac{1}{m} \cdot dZ_2 \cdot A_2^T$$

$$B_2 = B_2 - \text{alpha} * dB_2$$

$$dB_2 = \frac{1}{m} \cdot \text{sum}(dZ_2, 1)$$

$$W_1 = W_1 - \text{alpha} * dW_1$$

$$dZ_1 = W_2^T \cdot dZ_2 * f_1'(Z_1)$$

$$B_1 = B_1 - \text{alpha} * dB_1$$

$$dW_1 = \frac{1}{m} \cdot dZ_1 \cdot A_1^T$$

$$dB_1 = \frac{1}{m} \cdot \text{sum}(dZ_1, 1)$$